

Министерство образования и науки Российской Федерации
Саратовский государственный технический университет

**А.А. Сытник, Т.Э. Шульга,
Н.С. Вагарина, Н.И. Мельникова**

**ОСНОВЫ ПОСТРОЕНИЯ OWL-ОНТОЛОГИЙ
С ИСПОЛЬЗОВАНИЕМ РЕДАКТОРА
PROTÉGÉ DESKTOP 5.0 БЕТА**

Учебное пособие

Саратов 2015

УДК 004.738.52
ББК 32.973.202
О-75

Рецензенты:

Кафедра математической теории упругости и биомеханики
Саратовского государственного университета
имени Н.Г. Чернышевского
Начальник учебно-организационного отдела
Поволжского регионального центра новых информационных технологий
Саратовского государственного университета имени Н.Г. Чернышевского
М.Б. Орлова

Одобрено
редакционно-издательским советом
Саратовского государственного технического университета

О-75 Основы построения OWL-онтологий с использованием редактора
Protégé Desktop 5.0 beta: учеб. пособие / **А.А. Сытник,**
Т.Э. Шульга, Н.С. Вагарина, Н.И. Мельникова. Саратов:
Сарат. гос. техн. ун-т, 2015. 84 с.
ISBN 978-5-7433-2930-4

В учебном пособии описываются основные понятия и принципы онтологического моделирования в редакторе Protégé Desktop 5.0 beta. Приводятся различные определения термина онтология, описывается область применения онтологий, назначение основных элементов языка OWL2 процесс разработки онтологии, приводится инструкция по установке редактора онтологий Protégé Desktop 5.0 beta, способы создания основных элементов OWL-онтологий в редакторе Protégé.

Для студентов направлений «Прикладная информатика», «Программная инженерия» и «Информатика и вычислительная техника». Представляет интерес для преподавательского состава, интересующегося новейшими подходами, технологиями и разработками в области семантического веба.

УДК 004.738.52
ББК 32.973.202

ISBN 978-5-7433-2930-4

© Саратовский государственный
технический университет, 2015
© Сытник А.А., Шульга Т.Э.,
Вагарина Н.С., Мельникова Н.И., 2015

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	4
ГЛАВА 1. ОСНОВНЫЕ ПОНЯТИЯ ОНТОЛОГИЧЕСКОГО МОДЕЛИРОВАНИЯ.....	7
1.1. Понятие онтологии и области использования	7
1.2. Элементы OWL-онтологий	9
1.3. Процесс разработки онтологий	12
ГЛАВА 2. УСТАНОВКА PROTÉGÉ	16
ГЛАВА 3. ОСНОВЫ ПОСТРОЕНИЯ OWL-ОНТОЛОГИИ В PROTÉGÉ	18
3.1. Создание новой OWL-онтологии	18
3.2. Именованные классы	20
3.3. Непересекающиеся классы	22
3.4. Использование инструмента Create Class Hierarchy для создания классов	23
3.5. OWL-свойства	27
ГЛАВА 4. ОПИСАНИЕ ОБЪЕКТНЫХ СВОЙСТВ	30
4.2. Обратные свойства	31
4.3. Домен и диапазон свойств.....	32
4.4. Функциональные свойства.....	34
4.5. Транзитивные свойства	35
4.6. Симметричные свойства	36
4.7. Асимметричные свойства	37
4.8. Рефлексивные свойства.....	37
4.9. Иррефлексивные свойства	37
ГЛАВА 5. ОПИСАНИЕ И ОПРЕДЕЛЕНИЕ КЛАССОВ	39
5.1. Ограничения	39
5.2. Ограничения существования	40
5.3. Использование машины логического вывода	47
5.4. Необходимые и достаточные условия.....	51
5.5. Универсальные ограничения	54
5.6. Аксиомы замыкания	56
5.7. Паттерн проектирования «Разделение значений»	59
5.8. Ограничения кардинальности	67
ГЛАВА 6. СВОЙСТВА ТИПА ДАННЫХ И СОЗДАНИЕ ИНДИВИДОВ.....	72
6.1. Свойства типа данных.....	72
6.2. Создание индивидов	72
6.3. Ограничения на диапазон значений свойств типа данных	76
6.4. Ограничения на значения объектных свойств	78
6.5. Задание класса перечислением индивидов	79
СПИСОК ЛИТЕРАТУРЫ.....	81

ВВЕДЕНИЕ

Традиционно для человечества одним из основных способов хранения и передачи знаний считались тексты на естественных языках. Однако очевидно, что не любой текст может служить источником знаний. Прежде всего, это связано с большим количеством национальных языков народов, населяющих Землю (по оценкам ЮНЕСКО существует более 2700 языков, народов и народностей). Кроме того, многие проблемы в обмене и создании знаний связаны с неоднозначным или неадекватным восприятием смысла текста на естественном языке: в цепи передачи знаний отправитель и получатель знания часто пользуются различными представлениями (сформированными в результате предшествующего жизненного и профессионального опыта) и различной терминологией. Учитывая экспоненциальный рост объемов текстовой информации в последние десятилетия, к предыдущим двум ограничениям добавляется и ограничение физической невозможности прочитать все тексты по определенной предметной области даже на одном языке.

Возникает вопрос о разработке некоторых стандартных форм представления знаний (моделей знаний) [1], лишенных основного недостатка естественных языков в точки зрения анализа – многозначности. Предполагается, что такие формы представления знаний могут обрабатываться машинным способом.

Одной из популярных моделей знаний на сегодняшний день является онтология (Ontology). Онтологическое моделирование (Ontological Modeling) – процесс построения, развития, обработки и использования онтологии предметной области. С конца XX века онтологическое моделирование начало формироваться как новое научное направление, о чем свидетельствует большое количество научных работ в данной области и бурное развитие инструментария. Популярность данного подхода во многом объясняется тем, что он поддерживается в рамках концепции семантического веба [2].

Онтологическое моделирование развивается на пересечении ряда областей знаний, имеющих собственную богатую историю развития и научных исследований: философии, лингвистики, логики, концептуального моделирования (область знания, развивающаяся преимущественно в рамках теории баз данных и занимающаяся описанием сущностей, их структур, характерных соотношений между ними и их поведения в предметной области), инженерии знаний (Knowledge Engineering). Терминология этих областей используется в онтологическом

моделировании. Однако существует также и тенденция применять терминологию, специфическую для онтологического моделирования, к различным областям знаний. В целом терминологию в области онтологического моделирования до сих пор трудно назвать устоявшейся. В частности, русскоязычная терминология, помимо всего прочего, зависит от англоязычной как международной, и разночтения в ней возникают в результате разных переводов терминов с английского языка на русский [3]. Поэтому российскими учеными предпринята попытка создать глоссарий онтологического моделирования [4].

Как и любая модель знаний, онтология должна быть записана на определенном языке. Языки описания онтологий условно можно разделить на две категории в зависимости от их выразительной мощности: языки, частично реализующие формализмы для построения онтологий и языки в полной мере реализующие требуемые формализмы для разработки онтологий. К первой категории можно отнести языки UML, язык ER-диаграмм и язык RDFS. Ко второй категории относятся дескрипционные логики и язык OWL, языки CycL, Ontolingua, RuleML, KIF, Common Logic, DAML+OIL. Наиболее популярным языком представления онтологий, рекомендуемым консорциумом W3C (World Wide Web Consortium) для представления знаний в семантическом вебе, является язык OWL (Web Ontology Language) [5]. Процесс записи онтологии на этом языке (даже с помощью самых простых человекочитаемых синтаксисов) является очень трудоемким. Поэтому на настоящий момент разработано большое количество специальных программных средств для автоматизации данного процесса, называемых редакторами онтологий [6,7]. Одним из самых полнофункциональных средств данного класса является свободно распространяемый редактор с открытым кодом Protégé [8], разработанный в Стэнфордском университете.

В данном учебном пособии описываются основные понятия и принципы онтологического моделирования в редакторе Protégé Desktop 5.0 beta.

В первой главе пособия рассматриваются основные понятия онтологического моделирования: приводятся различные определения термина онтология, описывается область применения онтологий, назначение основных элементов языка OWL² (классов, свойств,

¹ В данном пособии не рассматриваются вопросы связи языка OWL с языком RDFS и моделью данных RDF, не описываются диалекты, профили и конкретные синтаксисы языка (Turtle, RDF/XML, OWL/XML и др.). Рекомендуем читателям изучить эти вопросы самостоятельно [10].

ограничений на классы и свойства, индивидов), процесс разработки онтологии.

Во второй главе пособия приводится инструкция по установке редактора онтологий Protégé Desktop 5.0 beta.

В главах 3-6 приводятся способы создания основных элементов OWL-онтологий (классов, свойств, аксиом, индивидов) в редакторе Protégé. В качестве сквозного примера в этих главах используется пример «Онтология пицц» из учебного пособия «A Practical Guide To Building OWL Ontologies Using Protege 4 and CO-ODE Tools Edition 1.3» [9]. Для получения навыков работы с редактором Protégé читателю предлагается выполнить ряд упражнений.

В пособии используются следующие обозначения.

Это определение

Это **Названия кнопок, вкладок** и других элементов программ.

Это *Имена элементов* онтологии.

Упражнение 1. Это название упражнения

1. Это сделай раз
2. Это сделай два

Глава 1. ОСНОВНЫЕ ПОНЯТИЯ ОНТОЛОГИЧЕСКОГО МОДЕЛИРОВАНИЯ

1.1. Понятие онтологии и области использования онтологий

К наиболее известным моделям представления знаний относят модели, построенные на основе исчисления высказываний и исчисления предикатов, продукционные модели, семантические сети (Куиллиан, 1966), фреймовые модели (Минский, 1974), ER-диаграммы (Чень, 1976), онтологии [11]. Таким образом, **онтология** – это модель представления знаний.

Прежде чем уточнить определение онтологии как модели представления знаний, заметим, что термин «онтология» активно используется в философии. Онтология (от др.-греч. ὄντος – сущее, то, что существует + λόγος – учение, наука) – учение о сущем, бытии как таковом, раздел философии, изучающий фундаментальные принципы бытия, его наиболее общие сущности и категории, структуру и закономерности. Основы данного учения были заложены в работах Парменида и Аристотеля, в нем ищутся ответы на следующие вопросы: что существует, что характеризует бытие, как классифицировать то, что существует. Сам термин «онтология» был введен в 1613 году Рудольфом Гоклениусом и в 1656 году Иоганом Клаубергом, предложившим его (в варианте «онтософия») в качестве эквивалента понятию «метафизика». В практическом употреблении термин был закреплён Христианом фон Вольфом, явно разделившим семантику терминов «онтология» и «метафизика».

В научных кругах, занимающихся инженерией знаний, самое популярное определение онтологии было дано в 1993 г. Томасом Грубером «**Онтология** – это явная спецификация концептуализации» [12]. Уточним данное определение [11]. «**Онтология** – это формальное, явное, точное определение (спецификация) совместно используемой концептуализации». Под концептуализацией понимается абстрактное упрощенное представление мира, которое формируется для некоторых целей. Онтология является точным определением (спецификацией), потому что она представляет концептуализацию в конкретной форме. Она является явной, потому что все используемые в ней ограничения явно определены. Слово «формальная» означает, что онтология должна быть машинно-обрабатываемой. Слово «совместно используемая» указывает на то, что онтология содержит согласованные знания.

Более приближенным к управлению знаниями можно признать определение, приведенное в [1]. «Онтология – это базы знаний специального типа, которые могут «читаться» и пониматься, отчуждаться от их разработчика и /или физически разделяться их пользователями». Однако данное определение не позволяет понять все особенности данной модели.

Консорциум W3C дает следующее определение. **Онтология** – это формальная модель представления знаний в некоторой предметной области. Онтология описывает типы объектов (классы – classes), взаимосвязи между ними (свойства – properties), и способы совместного использования классов и свойств (аксиомы – axioms) [13].

Еще более формальное определение дано в спецификации FIPA (Foundation for Intelligent Physical Agents). Онтология – это подробная спецификация структуры определенной предметной области, включающая словарь (список) логических констант и предикатных символов для описания предметной области и набор логических высказываний, формулирующих существующие в данной предметной области ограничения и определяющих интерпретацию словаря.

Таким образом, любая онтология содержит список имен сущностей (концептов, терминов, понятий, классов) в определённой предметной области и их значений (тезаурус) и набор связей между ними и их значений.

Использование онтологий позволит решить следующие основные задачи:

- выработка общего понимания терминов как между людьми, так и между программами (стандартизация знаний), что повышает совместное использование знаний;
- интеграция знаний из различных источников (если при описании данных использовать, например, одну онтологию предметной области, то интеграция данных может осуществляться автоматически);
- представление знаний о предметной области в явной форме (что, в свою очередь, облегчает понимание и поддержание знаний, способствует развитию знаний);
- повторное использование знаний (избавляет от необходимости «изобретать колесо», сокращает дублирование и противоречивость);
- анализ знаний (с целью повторного использования и получения новых знаний).

Область онтологического моделирования можно условно разделить на два направления: разработка онтологий и использование онтологий.

Разработка онтологий, в свою очередь, включает знания в трех областях: извлечение знаний, структурирование знаний, формализация знаний (языки онтологий, редакторы онтологий, системы хранения и публикации онтологий).

Перечислим основные области применения онтологий:

- семантический веб;
- поисковые системы (в т.ч. в интернете);
- системы обучения;
- создание баз знаний в различных системах искусственного интеллекта (экспертных системах, системах управления знаниями, интеллектуальных роботов);
- анализ и перевод текстов;
- системы интеграции информации.

1.2. Элементы OWL-онтологий

Язык OWL – полнофункциональный язык описания онтологий, предложенный консорциумом W3C в 2004 году. В 2012 появилась вторая версия данного языка – OWL2.

Напомним, что в соответствии с определением консорциума W3C онтология описывает

- типы объектов (**классы** – classes),
- взаимосвязи между ними (**свойства** – properties),
- способы совместного использования классов и свойств (**аксиомы** – axioms).

Кроме того, многие исследователи считают, что понятие онтологии подразумевает наличие конкретных сущностей (экземпляров) классов (**индивидов** – instances, individuals).

Рассмотрим данные элементы онтологии на языке OWL подробнее.

Основная цель любой онтологии – классифицировать объекты в точки зрения семантики. В языке OWL эта цель достигается за счет использования **механизма классов, иерархий классов и индивидов этих классов**, называемых экземплярами (instances – individuals).

Класс описывает группу сущностей (индивидов), которые имеют общие характеристики.

Например, класс *ЖилоеПомещение* описывает группу изолированных помещений, пригодных для постоянного проживания людей, класс *Квартира* описывает группу жилых помещений, состоящих

из одной или нескольких смежных комнат с отдельным наружным выходом, составляющих отдельную часть дома. Если некоторая сущность является членом класса, то это при машинной интерпретации означает, что она подпадает под семантическую классификацию, определяемую этим классом.

Основным отношением между классами в языке OWL является **иерархическое отношение «класс-подкласс»**.

Класс А является подклассом (subclass) класса В, если все экземпляры класса А являются экземплярами класса В.

Например, класс *Квартира* является подклассом класса *ЖилоеПомещение*, так как любая квартира является жилым помещением.

Индивид – конкретная сущность с собственной идентификацией¹.

С помощью средств языка OWL можно явным образом указать, что некий объект **является индивидом** некоторого класса. Например, *Квартира13* является индивидом класса *Квартира*. Кроме того, вывод о принадлежности индивида к конкретному классу может быть сделан **машиной логического вывода (reasoner)**² на основе ограничений на классы и свойств данного индивида.

Индивиды классов могут быть связаны друг с другом или с конкретными значениями данных с помощью двух типов **свойств**: объектных свойств и свойств типа данных.

- **Объектные свойства (Object Property)** – свойства, которые связывает индивиды двух классов. Например, индивиды класса

¹ В последних версиях стандартов семантического веба для идентификации любых ресурсов (классов, свойств, индивидов) используется IRI. IRI (Internationalized Resource Identifier) – интернационализированный идентификатор ресурса, представляет собой Unicode-строку и соответствует синтаксису, определённом в стандарте URI. URI (Uniform Resource Identifier) представляется собой либо URN (Uniform Resource Name), например ISBN книги, либо URL (Uniform Resource Locators), например адрес веб-документа http://www.w3.org/standards/techs/linkedata#w3c_all

² Машина логического вывода при онтологическом моделировании выполняет две основные функции:

- проверяет онтологию на согласованность (непротиворечивость),
- осуществляет автоматическое построение иерархий классов и классификацию индивидов на основе заданных разработчиком иерархий классов, ограничений и описаний индивидов.

Человек могут быть связаны свойством *приживаетВ* с индивидами класса *Квартира*).

- **Свойства типа данных (Datatype Property)** – свойства, которые связывают сущности классов с литералами (т.е. фиксированными значениями строкового или числового типа). Например, индивиды класса *Квартира* могут быть связаны свойством *имеетНомер* с целым числом.

Таким образом, подразумевается, что справедливо следующее.

Любое свойство в онтологии имеет область значений (**домен**), т.е. класс, индивиды которого будут связываться этим свойством и область определения (**диапазон**), т.е. класс, с индивидами которого будет связываться это свойство или тип данных.

Например, для свойства *проживаетВ* доменом является класс *Человек*, а диапазоном – класс *Квартира*, а для свойства *имеетНомер* доменом является класс *Квартира*, а диапазоном – целый тип данных.

Свойства, так же как и классы, могут быть организованы в иерархии.

Свойство А является **подсвойством** (subproperty) класса В, если все индивиды, связанные свойством А, связаны также и свойством В.

Например, свойство *арендует* является подсвойством свойства *проживаетВ*, так как если человек арендует жилое помещение, то этот человек и проживает в нем. Обратное не всегда верно. Например, человек может быть ребенком, проживающим в квартире с семьей, или просто гостем, который не платит арендную плату.

Обратите внимание, что свойства и классы в OWL имеют другую интерпретацию, чем в объектно-ориентированном программировании (ООП). В ООП класс имеет определённый набор свойств, и все экземпляры этого класса также имеют этот же набор свойств с определенными значениями. В OWL свойства определяются независимо от класса. Таким образом, два индивида одного класса могут иметь разные свойства и наоборот, индивиды разных классов могут иметь одно свойство. Таким образом, если в ООП основной акцент ставится на управлении классами, то в OWL – на управлении свойствами. Это дает возможность создания очень гибких, расширяемых онтологий. Фактически любой разработчик может внести в онтологию свое свойство (т.е. свое описание

существующих ресурсов), и, таким образом, реализуется основной принцип веба: «Кто угодно может сказать, что угодно о чем угодно!».

На свойства и классы могут быть наложены различные ограничения (**аксиомы**).

Например, в онтологии можно явным образом указать, что квартира может находиться только в одном доме, а дом либо не содержит ни одной квартиры, либо содержит две и более квартир или что если комната *A* находится в квартире *B*, а квартира *B* находится в доме *C*, то из этого следует, что комната *A* находится в доме *C*.

В заключение заметим, что, кроме описанных выше элементов (классов, свойств, ограничений на классы и свойства, индивидов), онтология обычно содержит аннотационные свойства, которые используются для добавления человеко-читаемых меток, комментариев или пояснений для OWL-онтологий, классов, свойств и индивидов. Свойства аннотаций игнорируются машинами вывода.

1.3. Процесс разработки онтологий

Ручная разработка онтологий является трудоемкой задачей, требующей существенных затрат времени, ресурсов и привлечения высококвалифицированных специалистов. У разработчика онтологии должны быть достаточные знания в предметной области, для которой строится онтология, либо свободный доступ к эксперту, обладающему такими знаниями.

Можно выделить следующие основные этапы в процессе разработки онтологии [10, 14].

1. Определение области и масштаба онтологии.
2. Рассмотрение возможности повторного использования существующих онтологий.
3. Перечисление основных терминов онтологии.
4. Определение иерархии классов и свойств.
5. Определение аксиом классов и свойств и проверка онтологии на согласованность
6. Определение индивидов.
7. Проверка онтологии с индивидами на согласованность.

Создание онтологии, как и любой другой процесс разработки, на практике реализуется не линейно: этапы разработки могут повторяться и на любой стадии может возникнуть необходимость возврата на более ранние этапы. Рассмотрим отдельные шаги разработки, приведенные выше.

Определение области и масштаба онтологии

Создание онтологии предметной области не является самоцелью. Разработка онтологии, как и разработка любой структуры данных, необходима для использования ее другими приложениями. Другими словами, онтология является моделью конкретной предметной области, которая разрабатывается для конкретной цели. Следовательно, не существует *правильной* онтологии определенной предметной области, онтология всегда имеет жизнеспособные альтернативы. Что именно будет включено в эту абстракцию, определяется в зависимости от задач приложения, для которого она разрабатывается, и от будущих потенциальных вариантов ее использования. На данном шаге должны быть даны ответы на следующие основные вопросы. Для какой предметной области разрабатывается онтология? Для чего планируется использовать онтологию? На какие типы вопросов должна давать ответы информация в онтологии? Кто будет использовать и поддерживать онтологию?

Рассмотрение возможности повторного использования онтологий

Разработку онтологии редко приходится начинать «с нуля». Почти всегда можно найти уже существующую онтологию, которую можно рассматривать, по крайней мере, как отправную точку при разработке собственной онтологии.

Для поиска существующих онтологий можно пользоваться следующими инструментами:

- Linked Open Vocabularies (каталог открытых связанных словарей); <http://lov.okfn.org/dataset/lov/>;
- Сервис «Статистика LOD» <http://stats.lod2.eu/vocabularies>
- Sindice, the Semantic Web Index (поисковая система в семантическом вебе) <http://sindice.com/main/about>;
- Сервис SameAs.org (<http://www.sameas.org>) для поиска эквивалентных IRI;
- Каталог связанных данных DataHub <http://datahub.io/>;
- Проект Swoogle <http://swoogle.umbc.edu/>.

В целом вопрос импорта онтологий и создания отображения между ними по-прежнему остается открытым и рассматривается как одна из самых серьезных проблем семантического веба.

Перечисление терминов онтологии

Первым шагом непосредственно при определении онтологии является создание неструктурированного списка, который содержит все возможные понятия, которыми должна оперировать онтология. Как правило, имена существительные в этом списке образуют основу для имен классов, а глаголы (или выражения с глаголами) – для имен свойств (например, «является частью», «имеет компонент»).

На данном этапе могут быть продуктивно использованы традиционные инструменты инженерии знаний, такие как лэддеринг (метод лестницы – метод проведения глубинных интервью, при котором задаваемые вопросы сначала касаются характерных особенностей явления, продукта, а потом и черт самого пользователя) и грид-анализ (анализ, основанный на матрицах решений). Данные методы позволяют получить как сам список терминов, так и их первоначальную структуру.

Определение иерархии классов и свойств

После определения классов и свойств онтологии их необходимо организовать в иерархию. Нельзя дать однозначной рекомендации по поводу того, как лучше проектировать иерархии классов и свойств: нисходящим или восходящим методом.

Важно убедиться, что построенная иерархия представляет собой таксономию (т.е. классы связаны в ней отношением «класс-подкласс»).

Определение свойств часто выполняется параллельно с определением классов: естественно определять свойства, которые связывают классы, именно при определении классов и организации этих классов в иерархию. При определении свойств и соотнесении их с определенными классами имеет смысл сразу указывать их домен и диапазон. С одной стороны, из-за особенности отношения «класс-подкласс» имеет смысл относить свойства к самому старшему классу в иерархии, к которому их только возможно отнести, в этом случае их можно будет использовать для всех младших членов этой иерархии. С другой стороны, иногда полезно определить в качестве домена и диапазона свойства, как можно более узкие классы. Это позволит выявлять потенциальную несогласованность онтологии путем нахождения противоречий в соответствующих доменах и диапазонах.

Определение аксиом классов и свойств и проверка онтологии на согласованность

На этом шаге накладываются ограничения на свойства (основные типы ограничений на свойства описаны в главе 2) и ограничения на совместное использование классов и свойств (главы 3-4).

После выполнения этого шага в процессе разработки онтологии возможно проверить онтологию на внутреннюю согласованность. До реализации этого шага такая проверка была невозможна, механизмы иерархий классов и свойств сами по себе не позволяют выразить противоречивость знаний. Проверка на противоречивость в современных редакторах онтологий осуществляется автоматически с помощью машины логического вывода.

Определение индивидов

Разработка онтологии, так же как, например, и структуры базы данных, осуществляется с целью ее последующего использования в одном или нескольких программных приложениях. Для того чтобы онтология была применима на практике, ее надо наполнить корнетами индивидами и установить связи (свойства) между этими индивидами. Обычно число индивидов в онтологии на несколько порядков больше, чем число классов, на практике их число может достигать сотен тысяч. Поэтому наполнение онтологии на практике не осуществляется вручную. Для этого используют специальные программы, называемые конверторами. Наиболее развиты на данный момент конверторы данных их электронных таблиц и реляционных баз данных¹.

Проверка онтологии с индивидами на согласованность

После наполнения онтологии экземплярами необходимо повторить проверку онтологии на согласованность. Это также осуществляется с помощью машины логического вывода. Однако если предыдущая проверка была направлена на выявление аномалий в структуре классов и свойств с учетом заданных ограничений, то эта – на выявление аномалий в классификации индивидов. Например, противоречивость возникнет, если явно указано, что некий индивид является объектом некоторого класса, но при этом он не удовлетворяет ограничениям на этот класс.

¹ <http://www.w3.org/wiki/ConverterToRdf>

Глава 2. УСТАНОВКА PROTÉGÉ

Protégé является редактором онтологий и фреймворк для построения интеллектуальных систем. Данный продукт свободно распространяется и имеет открытую, легко расширяемую архитектуру. На момент написания данного учебного пособия количество пользователей данного продукта превышало 244 тыс.

На официальном сайте продукта [8] доступны две версии продукта: WebProtégé и Protégé Desktop. В данном пособии излагаются принципы работы с Protégé Desktop.

Protégé Desktop является полнофункциональной средой для разработки онтологий на язык OWL2, обеспечивающий возможность подключения известных машин логического вывода, таких как HermiT и Pellet.

Перечислим особенности редактора Protégé Desktop:

- соответствует стандартам консорциума W3C, в частности поддерживает различные синтаксисы языка OWL2 для сохранения онтологий;
- имеет настраиваемый пользовательский интерфейс;
- позволяет одновременно работать с несколькими онтологиями;
- обеспечивает визуальную поддержку онтологического моделирования;
- поддерживает рефакторинг онтологий, в т.ч. слияние онтологий, перемещение аксиом из онтологии в онтологию, автоматическое переименование объектов во всей онтологии;
- позволяет подключать машины логического вывода, а следовательно, обеспечивает автоматизированную проверку онтологий на согласованность и автоматическую классификацию сущностей и построение иерархий классов;
- обеспечивает возможность подключения широкого класса плагинов;
- является совместимым с редактором WebProtégé.

Для того чтобы установить Protégé Desktop в операционной системе Windows, выполните следующую последовательность шагов¹.

1. Скачайте программу с официального сайта Protégé: <http://protege.stanford.edu/>. Для этого на главной странице сайта нажмите **Download now (Загрузить сейчас)** и затем выберите **Protégé Desktop** и **Download for Windows**, заполните регистрационную форму и загрузите zip-файл на свой компьютер.

¹ Инструкция для установки в других операционных системах приведена на сайте: http://protegewiki.stanford.edu/wiki/Install_Protege5

2. Распакуйте zip-файл, например, в папку **C:/Program Files**. При распаковке автоматически создастся папка **Protege_5.0_beta**.

3. Откройте папку **Protege_5.0_beta** (рис. 2.1). Найдите файл **run.bat** и создайте для него ярлык (для этого выделите файл и из контекстного меню выберите команду **Создать ярлык**). Созданный ярлык с именем **run-Ярлык** можно поместить, например, на рабочий стол.

4. Переименуйте ярлык, например **Protege 5.0 beta**.

5. Воспользуйтесь ярлыком для запуска программы. Обратите внимание, что в момент запуска пакетный файл начнет покомандно исполняться (рис. 2.2). Этот процесс может занять несколько секунд. Ничего не предпринимайте. Дождитесь появления приветственного диалогового окна Protégé.

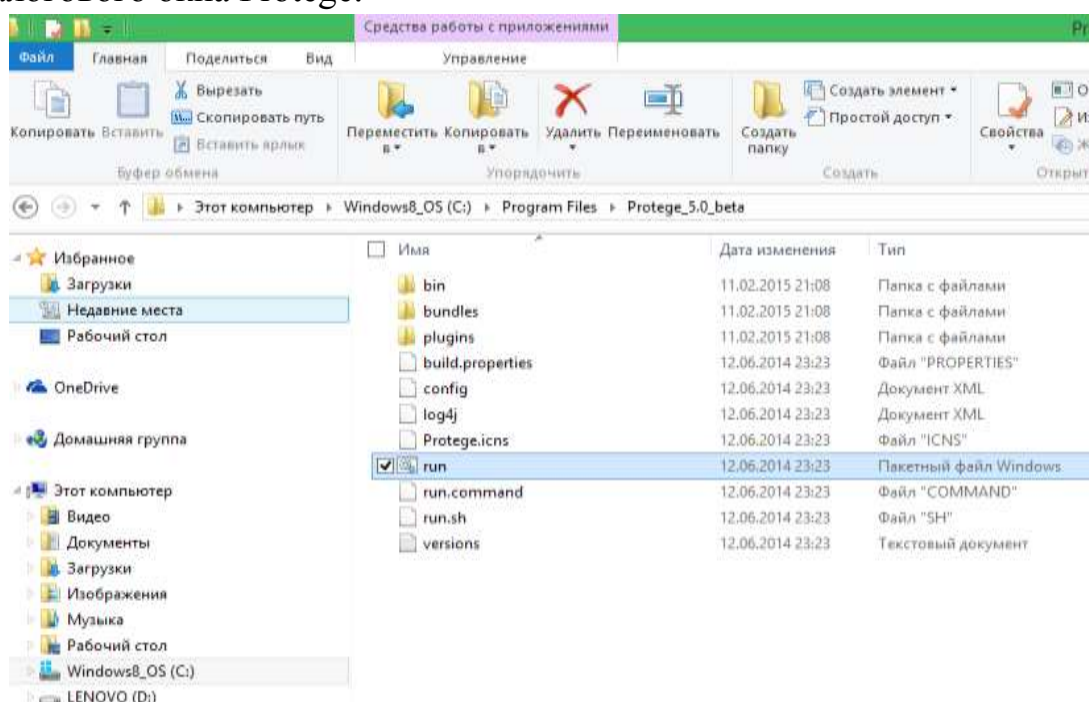


Рисунок 2.1. Папка Protege_5.0_beta

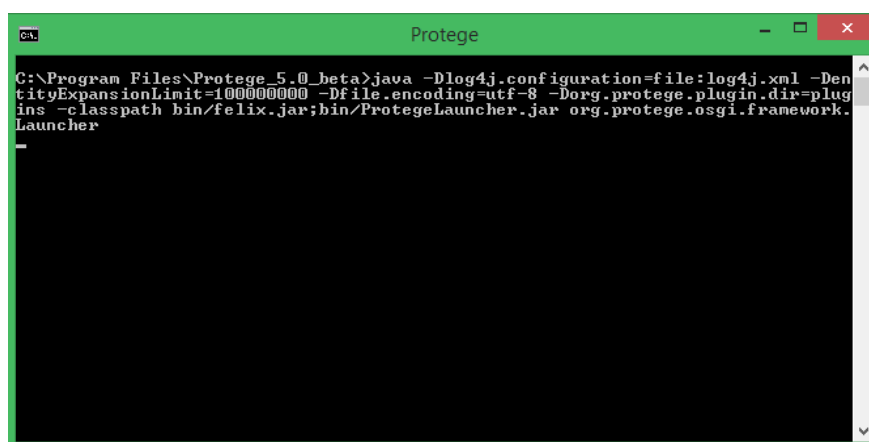


Рисунок 2.2. Запуск программы Protégé

Глава 3. ОСНОВЫ ПОСТРОЕНИЯ OWL-ОНТОЛОГИИ В PROTÉGÉ

3.1. Создание новой OWL-онтологии

В этой и следующих главах описывается, как создать онтологию, описывающую различные виды пиццы [9]. Этот пример позволяет продемонстрировать большое число возможностей Protégé.

OWL-онтология в Protégé включает классы, свойства и индивиды классов. Заметим, что если вы использовали Protégé для создания фреймовой модели знаний, то следует учитывать, что классы, свойства и индивиды в OWL-онтологии примерно соответствуют классам, слотам и индивидам в фреймовой модели.

Перейдем к разработке онтологии.

Упражнение 1. Создание онтологии

1. В приветственном диалоговом окне Protégé нажмите **Create New OWL Ontology** (Создать новую OWL-онтологию). (При повторном запуске Protégé для создания и сохранения онтологии пользуйтесь пунктами меню **File New** (Создать новый файл) и **File Save** (Сохранить файл) соответственно).
2. Появится окно мастера создания IRI онтологии.
3. Замените IRI по умолчанию на IRI `http://www.example.com/пицца.owl` и нажмите **Next**.
4. Для сохранения онтологии на жёстком диске выберите папку, формат RDF/XML и затем нажмите **Finish**.
5. На вкладке **Active Ontology** (Активная онтология) вы увидите общее описание созданной онтологии (IRI, аннотацию, такие метрики как количество классов, свойств, индивидов и т.д.). На данный момент аннотация онтологии не заполнена, а все метрические показатели равны нулю (рис 3.1). Внимательно изучите содержимое данной вкладки.

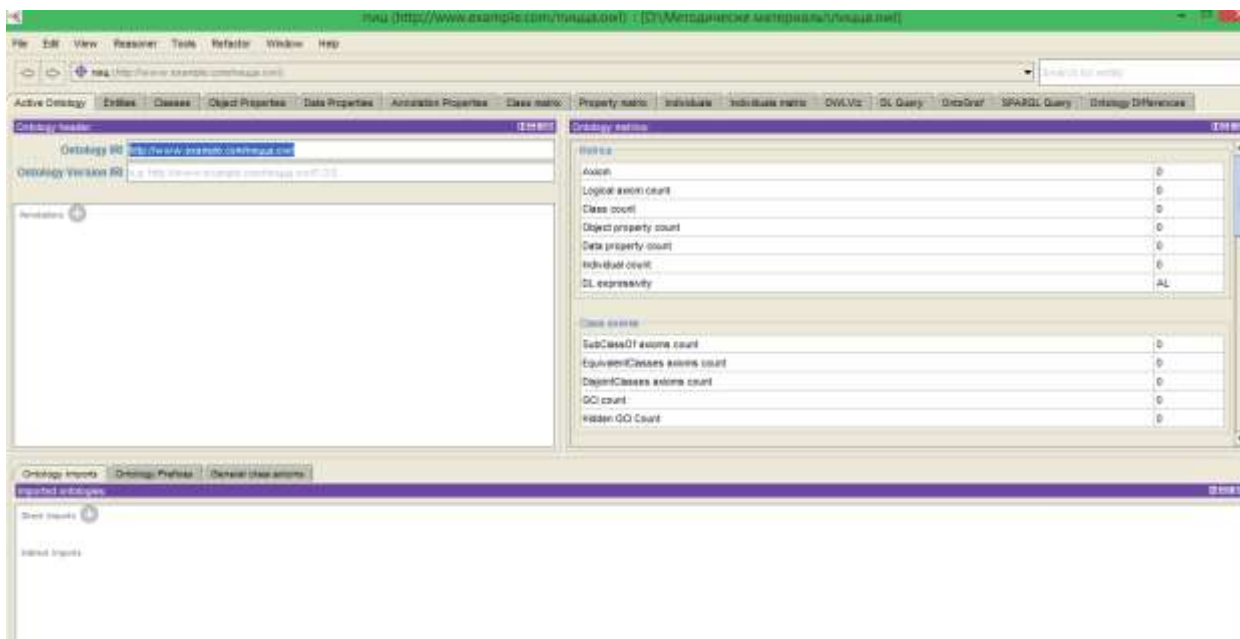


Рисунок 3.1. Вид вкладки Active Ontology

Упражнение 2. Добавления комментария к онтологии

1. Убедитесь, что выбрана вкладка **Active Ontology**.
2. Нажмите на иконку  (Добавить) рядом с **Annotation (Аннотация)**.
3. В левой части появившегося окна создания аннотации выберите из списка атрибутов атрибут **comment**.
4. В правой части введите комментарий «Онтология Пицца описывает варианты пицц с различными основами и начинками» (рис. 3.2) и нажмите **ОК**.

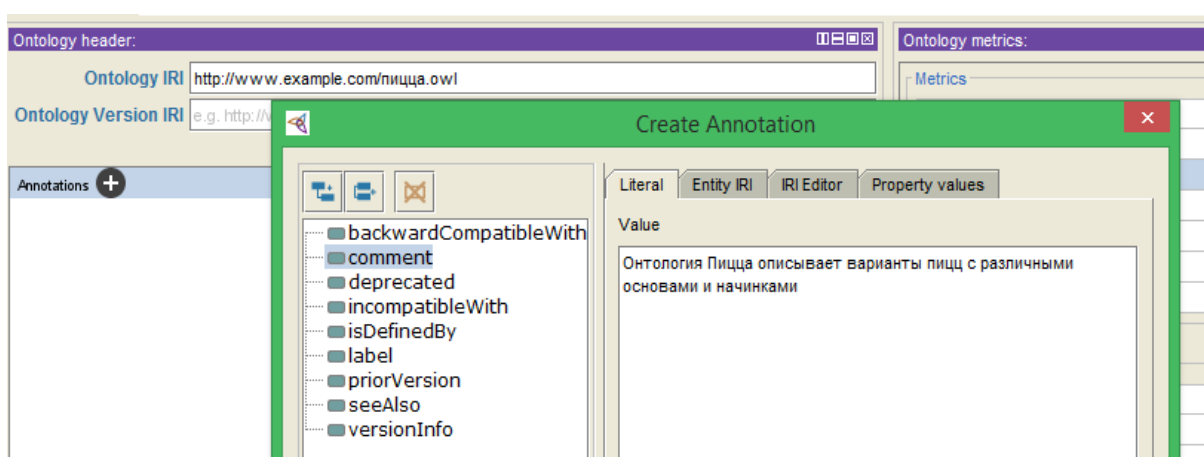


Рисунок 3.2. Окно создания аннотации онтологии

Заметим, что аннотирование онтологии и всех ее элементов является важным процессом при разработке онтологии, так как обеспечивает

человекочитаемые описания элементов, что, в свою очередь, облегчает взаимодействие между разработчиками и понимание онтологии пользователями.

3.2. Именованные классы

Как было отмечено выше, онтология включает классы. Именно классы являются основным строительным блоком OWL-онтологий. В редакторе Protégé 5.0 создание и редактирование классов осуществляется на вкладке **Classes (Классы)**. Для вновь созданной онтологии иерархия классов выглядит так, как показано на рис. 3.3, т.е. пустая онтология содержит один класс с именем *Thing*. Напомним, что OWL-классы рассматриваются как множество индивидов, т.е. множество объектов.

Класс *Thing* в OWL является обязательным классом любой онтологии, представляющим множество всех возможных объектов. Другими словами, все классы любой онтологии являются подклассами класса *Thing*.

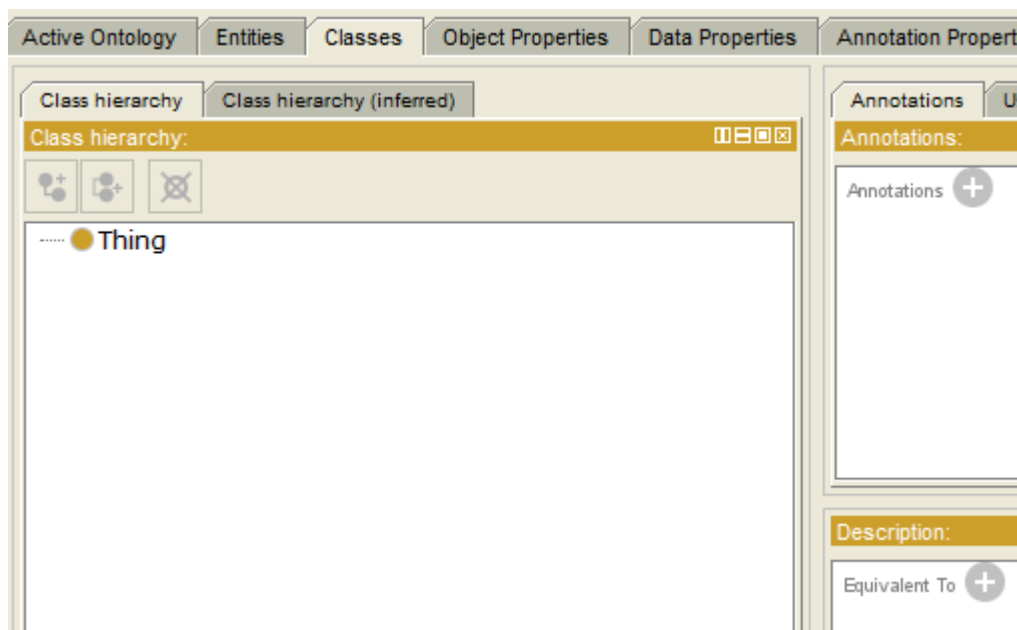


Рисунок 3.3. Вид вкладки Classes

Упражнение 3. Создание классов *Пицца*, *ПиццаНачинка*, *ПиццаОснова*

1. Убедитесь, что выбрана вкладка **Classes** и выделен класс *Thing*.
2. Нажмите на кнопку  (**Add subclass – Добавить подкласс**), показанную на рис. 3.3. С помощью этой кнопки создаётся новый

класс как подкласс выбранного класса. В данном случае при нажатии этой кнопки создаться подкласс класса *Thing*.

3. В появившемся окне создания нового класса (рис. 3.4) введите имя класса *Пицца* и нажмите **ОК**.
4. Повторите действия 1-3 для создания классов *ПиццаНачинка* и *ПиццаОснова*.

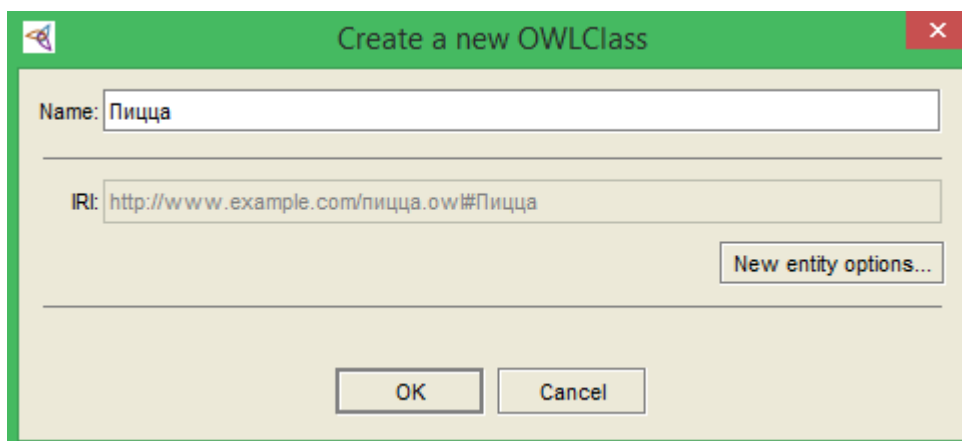


Рисунок 3.4. Окно создания нового класса

Созданная иерархия классов должна выглядеть так, как показано на рис. 3.5.

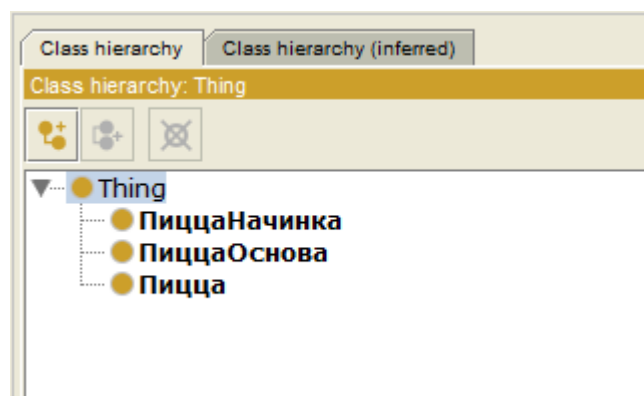


Рисунок 3.5. Начальная иерархия классов онтологии Пицца

Обратите внимание, что после создания класса *Пицца* для создания классов *ПиццаНачинка* и *ПиццаОснова* вам приходилось опять выделить класс *Thing* и только затем нажимать кнопку **Добавить подкласс**. Классы одного уровня иерархии можно создавать и другим способом, используют кнопку  **Add sibling class – Добавить одноуровневый класс**. Например, если выделен класс *Пицца*, можно нажать данную кнопку и создать класс *ПиццаНачинка*, если выделен класс *ПиццаНачинка*, можно таким же образом создать класс *ПиццаОснова*.

Если при создании классов вы допустили ошибку, выбрав одну из кнопок **Добавить подкласс** или **Добавить одноуровневый класс**, и класс оказался не на нужном уровне иерархии, то можно простым «перетаскиванием» его имени поместить его на нужный уровень.

Иерархия классов также называется **таксономией**.

Хотя не существует формальных правил именования OWL-классов, рекомендуется называть класс именем существительным в единственном числе. При этом желательно, чтобы имя класса начиналось с прописной буквы и не содержало пробелов, например, *Пицца*, *ПиццаНачинка*, *ПиццаОснова*. Альтернативным способом наименования классов может быть разделение слов в имени класса знаком подчёркивания, например *Пицца_Начинка*, *Пицца_Основа*. Неважно какую из двух предложенных нотаций для именования классов вы будете использовать, важно придерживаться в именовании имён одной выбранной нотации.

Если вы ошиблись в имени класса, то переименовать его можно, только выделив его и используя меню **Refactor⇒Rename entity (Переименовать сущность)**. Отсутствие стандартного способа переименования любого объекта в онтологии объясняется тем, что данный объект может участвовать в различных отношениях с другими объектами, и его имя должно быть изменено во всех этих отношениях.

3.3. Непересекающиеся классы

После того, как классы созданы, надо решить, являются ли они непересекающимися.

Два класса *A* и *B* являются непересекающимися, если любой индивид класса *A* не является индивидом класса *B*, а любой индивид *B* не является индивидом класса *A*.

Очевидно, что классы *Пицца*, *ПиццаНачинка*, *ПиццаОснова* являются непересекающимися.

Для того, чтобы указать, что два класса являются непересекающимися, надо выделить один из классов и нажать кнопку  (Добавить) рядом с **Disjount with (Не пересекается с)** области окна **Description (Описание)** (рис 3.6).

Упражнение 4. Указание, что классы *Пицца*, *ПиццаНачинка*, *ПиццаОснова* являются непересекающимися

1. Выделите класс *Пицца*.

2. Нажмите кнопку  (Добавить) рядом с **Disjount with** области окна **Пицца:Description** (рис. 3.6).
3. В появившемся окне выберите класс *ПиццаОснова* и нажмите **ОК**.
4. Повторите шаги 2-3, выбрав класс *ПиццаНачинка*.
5. Выделите класс *ПиццаНачинка*. Вы увидите, что он уже является непересекающимся к классом *Пицца*.
6. Для класса *ПиццаНачинка* повторите шаги 2-3, выбрав класс *ПиццаОснова*.

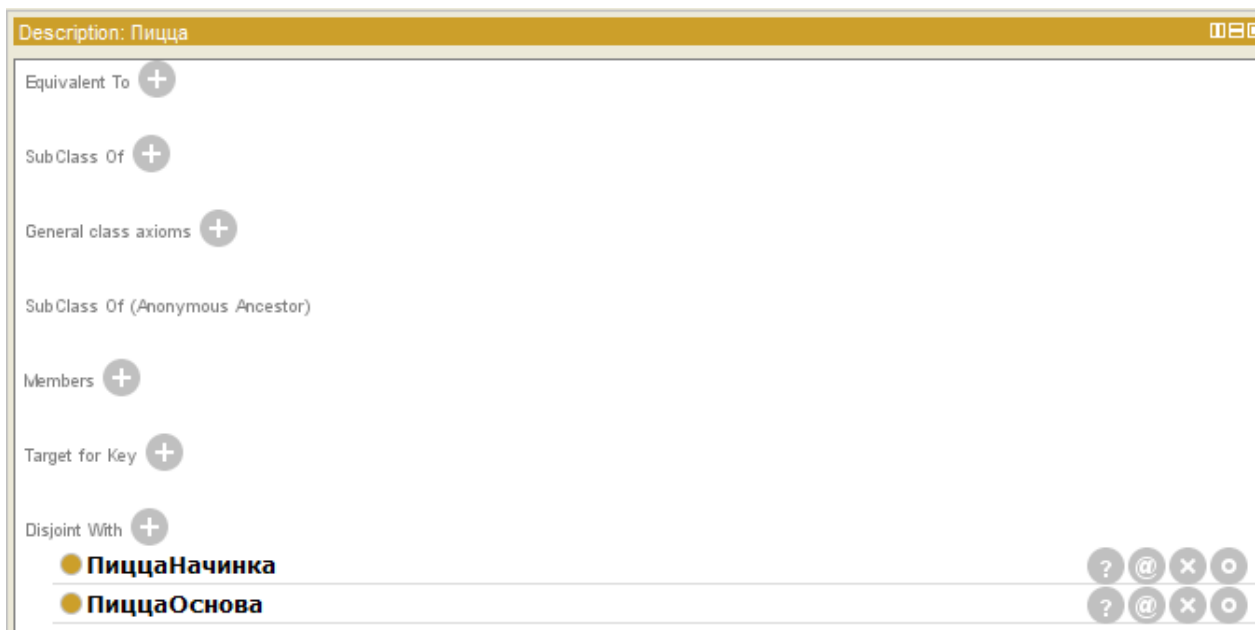


Рисунок 3.6. Область окна Description

Указывая, что классы являются непересекающимися, вы создаете аксиому (ограничение на классы), которая будет использоваться машиной вывода при проверке онтологии на согласованность. Например, если один и тот же индивид будет одновременно указан как индивид класса *Пицца* и индивид класса *ПиццаНачинка* (при том, что классы *Пицца* и *ПиццаНачинка* — непересекающиеся), то машина сделает вывод, что в онтологии существует аномалия.

3.4. Использование инструмента Create Class Hierarchy для создания классов

Процесс создания иерархии классов можно упростить за счет применения инструмента **Create Class Hierarchy** (Создать иерархию классов).

Упражнение 5. Создание подклассов класса *ПиццаОснова* с помощью инструмента **Create Class Hierarchy**

1. В иерархии класс выберите класс *ПиццаОснова*.
2. Выберите пункт основного меню программы **Tools** ⇒ **Create Class Hierarchy**.
3. В появившемся окне (рис. 3.7) убедитесь, что выделен класс *ПиццаОснова* и нажмите кнопку **Continue (Продолжить)**.
4. Появившемся окне в поле **Prefix** введите слово *Основа*, а большом текстовом поле на разных строках имена новых подклассов *Тонкая* и *Толстая* (рис 3.8), которые будут в дальнейшем дополнены указанным префиксом. Нажмите кнопку **Continue**.
5. В появившемся окне убедитесь, что отмечен пункт **Make sibling classes disjoint (Сделать одноуровневые классы непересекающимися)**. Нажмите кнопку **Finish**.
6. Убедись, что классы *ОсноваТонкая* и *ОсноваТолстая* появились в иерархии классов и являются непересекающимися.

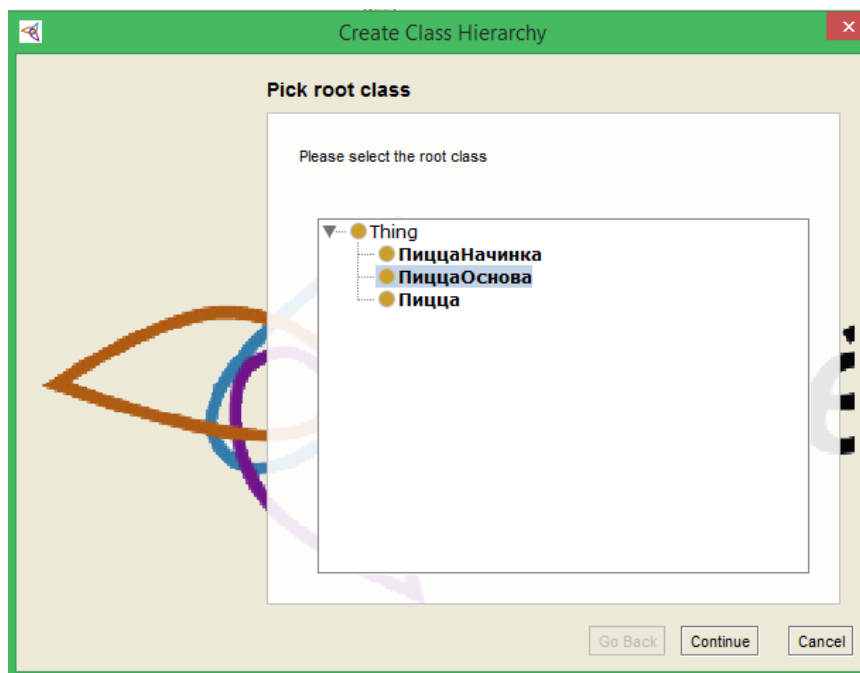


Рисунок 3.7. Окно **Create Class Hierarchy**: выбор класса

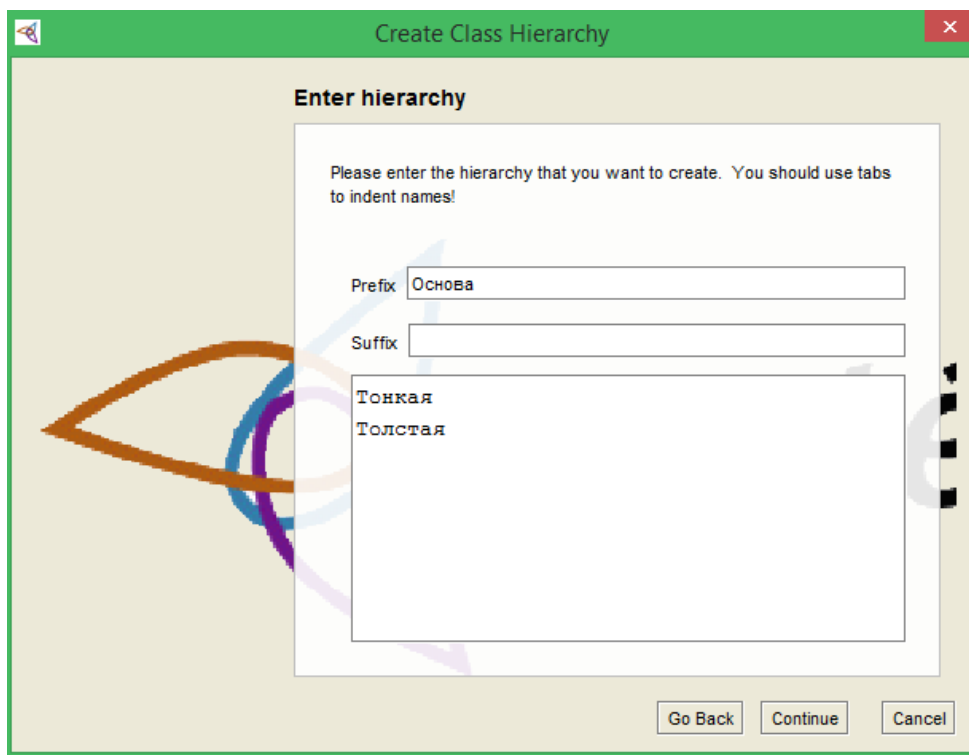


Рисунок 3.8. Окно Create Class Hierarchy: выбор имён подклассов

Упражнение 6. Создание подклассов класса ПиццаНачинка

1. В иерархии классов выберите класс *ПиццаНачинка*.
2. Выберите пункт основного меню программы **Tools** ⇒ **Create Class Hierarchy**
3. В появившемся окне убедитесь, что выделен класс *ПиццаНачинка* и нажмите кнопку **Continue**.
4. В качестве суффикса (suffix) имён в создаваемой иерархии укажите *Начинка*.
5. Для того чтобы определить уровень классов в создаваемой иерархии нужно пользоваться кнопкой табуляции. Введите имена классов как показано на рис 3.9. Нажмите кнопку **Continue**.
6. В появившемся окне убедитесь, что отмечен пункт **Make sibling classes disjoint** (Сделать одноуровневые классы непересекающимися). Нажмите кнопку **Finish**.
7. Убедись, что созданные классы появились в иерархии классов и являются непересекающимися (рис. 3.10)

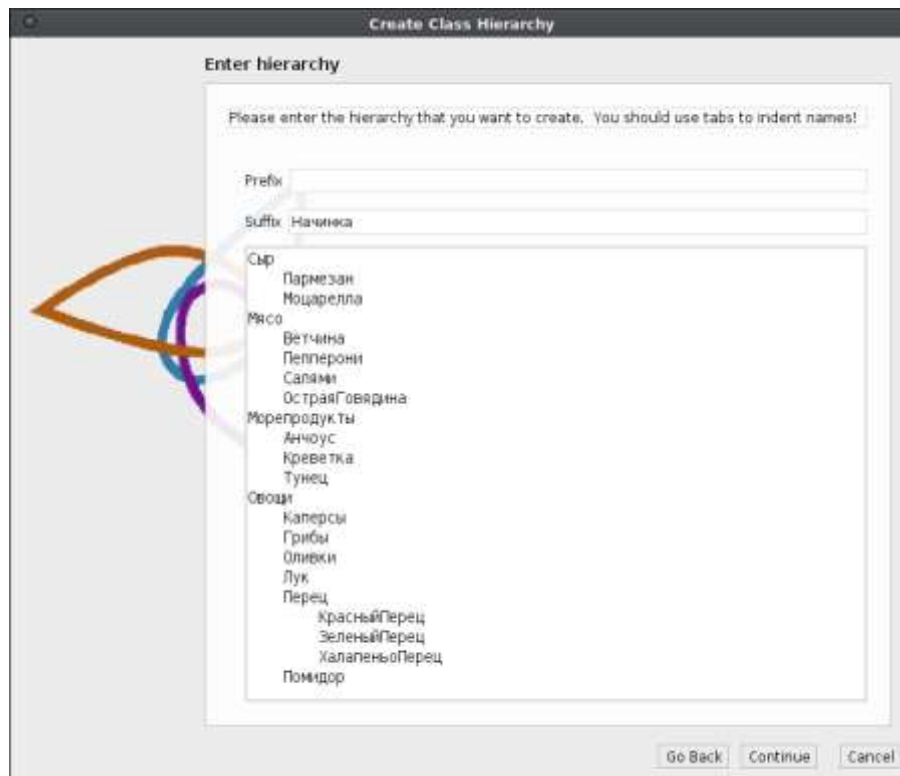


Рисунок 3.9. Иерархия классов начинки пиццы

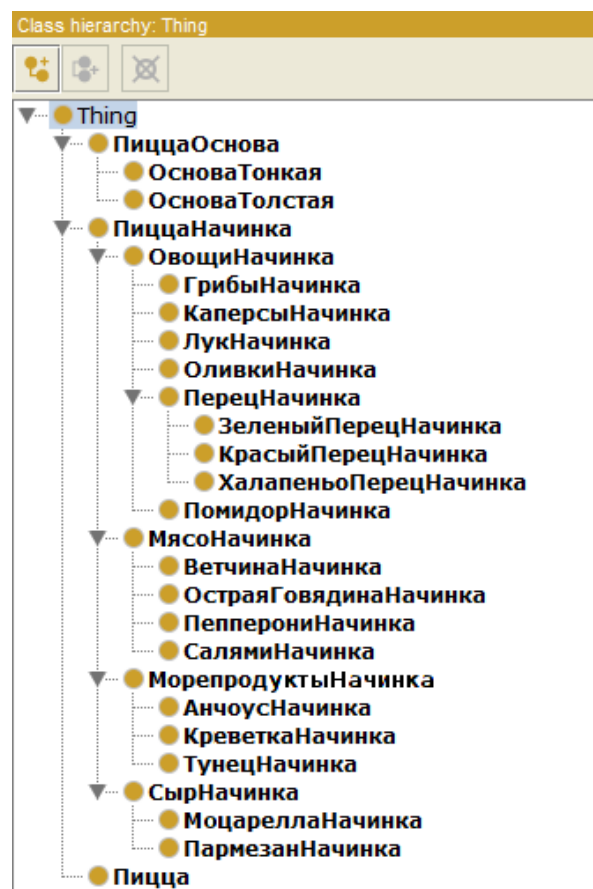


Рисунок 3.10. Созданная иерархия классов

3.5. OWL-свойства

Как было указано в первой главе, свойства представляют собой отношения. Выделяют два типа свойств: объектные свойства (Object Property) и свойства типа данных (Datatype Properties). Объектные свойства представляют собой отношение между двумя индивидами (объектами). Именно они описаны в этом разделе. Свойства типа данных описаны в главе 6.

В данном пособии фраза «индивид *A* связан свойством *P* с индивидом *B*» или «свойство *P* связывает индивид *A* и индивид *B*» означает, что справедливо следующее утверждение об индивидах «*A P B*». Например, фраза «индивид *Ольга* связан свойством *являетсяМатерью* с индивидом *Елена*» означает, что справедливо утверждение «*Ольга являетсяМатерью Елены*».

В общем случае свойства представляют собой несимметричные отношения, т.е. из утверждения «индивид *A* связан свойством *P* с индивидом *B*» нельзя сделать вывод о справедливости (или несправедливости) утверждения «индивид *A* связан свойством *P* с индивидом *B*». Например, если свойство *являетсяМатерью* не описано с помощью специальных характеристик (см. главу 4), то из утверждения «*Ольга являетсяМатерью Елены*» нельзя сделать вывод о справедливости (или несправедливости) утверждения «*Елена являетсяМатерью Ольги*».

В редакторе Protégé объектные свойства объектов создаются с помощью вкладки **Object Properties** (рис. 3.11).

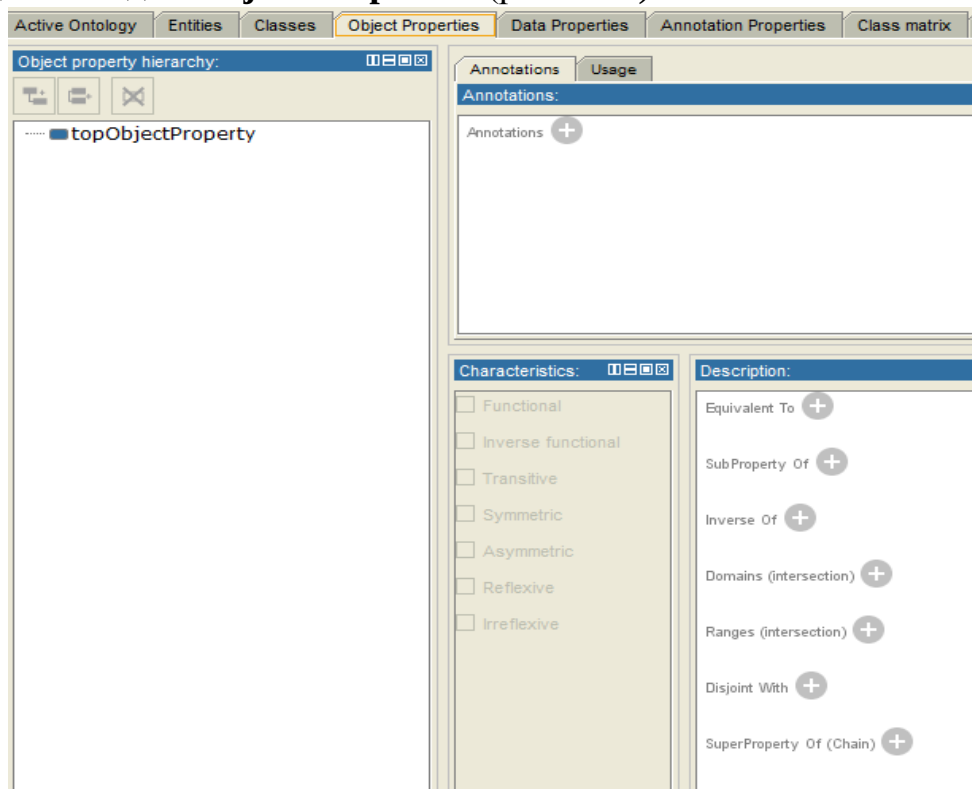


Рисунок 3.11. Вкладка Object Properties

В OWL свойства могут иметь подсвойства и таким образом образовывать, так же как и классы, иерархию. Обратите внимание, что вершиной в иерархии свойств является свойство *topObjectProperty*.

На рис. 3.12 показаны кнопки, расположенные в верхнем левом углу вкладки **Object Properties**, предназначенные для создания и удаления свойств.

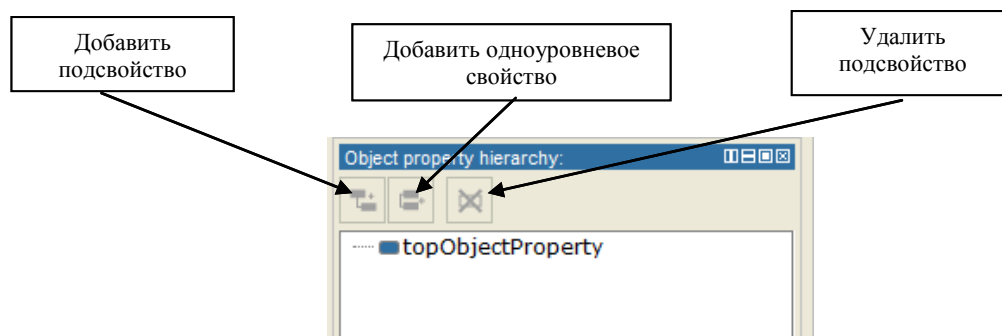


Рисунок 3.12. Кнопки создания и удаления свойств

Упражнение 7. Создание объектного свойства *имеетИнгредиент*

1. Переключитесь на вкладку **Object Properties**. Выберите свойство *topObjectProperty*.
2. Нажмите кнопку  (**Add sub property – Добавить подсвойство**).
3. В появившемся диалоговом окне введите имя нового свойства: *имеетИнгредиент* и нажмите **ОК**.

Хотя не существует формальных правил именования OWL-свойств, рекомендуется начинать название свойства с глагола со строчной буквы, а остальные слова в имени свойства начинать с большой буквы и указывать без пробелов. При этом рекомендуется в качестве префикса имени использовать такие слова как *является*, *имеет*, например, *имеетЧасть*, *являетсяЧастью*, *имеетПроизводителя*, *являетсяПроизведенным*.

Упражнение 8. Создание подсвойств свойства *имеетИнгредиент*: *имеетОснову* и *имеетНачинку*

1. В иерархии свойств выберите свойство *имеетИнгредиент*.
2. Нажмите кнопку  **Добавить подсвойство**, введите имя нового подсвойства *имеетОснову* и нажмите **ОК**.

3. Повторите действия 1-2 для создания подсвойства *имеетНачинку*.
4. Убедитесь, что созданная иерархия выглядит так, как показано на рис. 3.13.

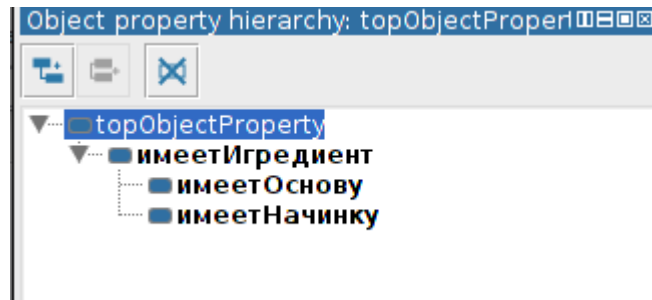


Рисунок 3.13. Иерархия свойств

Так же как и в случае с классами, для добавления свойства в иерархию можно использовать кнопку  **Add sibling property (Добавить одноуровневое свойство)**, и «перетаскивать» свойства внутри иерархии.

Глава 4. ОПИСАНИЕ ОБЪЕКТНЫХ СВОЙСТВ

В OWL можно для каждого свойства, так же как для класса задавать определённое описание. В Protégé это можно сделать с помощью области окна **Description** вкладки **Object Properties** (рис 4.1). В этой главе будут рассмотрены такие элементы описания как обратные (инверсивные) свойства, домен и диапазон свойств.

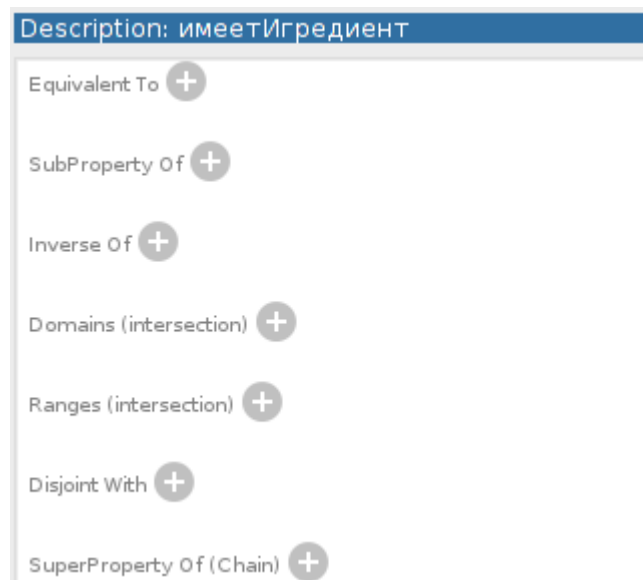


Рисунок 4.1. Область окна Description вкладки Object Properties

Кроме того, OWL позволяет придавать свойствам различный смысл за счёт использования так называемых характеристик свойств (аксиом свойств или ограничений на свойства).

Для того чтобы в Protégé указать одну или более характеристик свойства, надо выбрать его в иерархии свойств и затем отметить нужные характеристики в области окна **Characteristics (Характеристики)** (рис. 4.2).

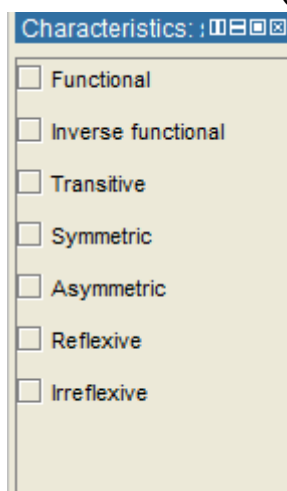


Рисунок 4.2. Область окна Characteristics вкладки Object Properties

В этой главе также рассматриваются различные характеристики свойств, а именно функциональность, транзитивность, симметричность и асимметричность, рефлексивность и иррефлексивность.

4.2. Обратные свойства

Каждое свойство в онтологии может иметь соответствующее ему обратное (инверсивное) свойство.

Если некоторое свойство связывает индивид *A* с индивидом *B*, то обратное ему свойство связывает индивид *B* с индивидом *A*.

Приведём примеры пар обратных свойств: *являетсяЧастью* и *имеетЧасть*, *являетсяПроизводителем* и *имеетПроизводителя*. В последнем случае, если продукт *A* связан свойством *имеетПроизводителя* с производителем *B*, то производитель *B* должен быть связан свойством *являетсяПроизводителем* с продуктом *A*.

В случае онтологии пицц очевидно, что для свойств *имеетИнгредиент*, *имеетНачинку* и *имеетОснову*, надо создать обратные свойства *являетсяИнгредиентом*, *являетсяНачинкой* и *являетсяОсновой* соответственно.

Для того, чтобы для заданного свойства создать обратное ему свойство, надо выделить свойство и нажать кнопку  (**Добавить**) рядом с **Inverse of (Обратное для)** области окна **Description** (рис 4.1).

Упражнение 9. Создание обратных свойств

1. В иерархии классов выберите класс *имеетИнгредиент* и создайте одноуровневое свойство *являетсяИнгредиентом*.
2. Для свойства *являетсяИнгредиентом* создайте два подсвойства *являетсяНачинкой* и *являетсяОсновой*.
3. Выберите свойство *являетсяИнгредиентом* и нажмите кнопку  (**Добавить**) рядом с **Inverse of** области окна **Description**.
4. В появившемся окне выберите обратное свойство *имеетИнгредиент* и нажмите **ОК**.
5. Повторите два раза шаги 3-4 для создания обратных свойств *имеетНачинку* и *имеетОснову* для свойств *являетсяНачинкой* и *являетсяОсновой* соответственно.

6. Убедитесь, что иерархия свойств выглядит так, как показано на рис. 4.3, и каждое свойство в этой иерархии имеет соответствующее обратное свойство.

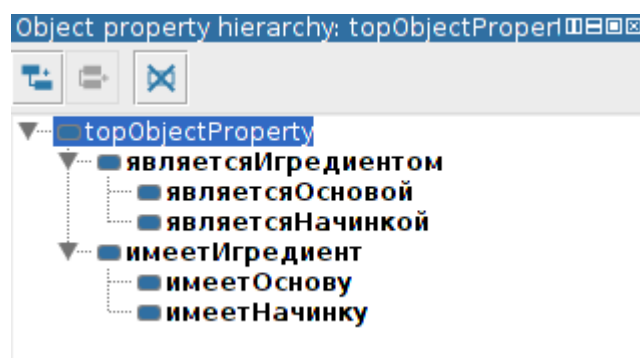


Рисунок 4.3. Иерархия свойств

4.3. Домен и диапазон свойств

Свойства могут иметь домен (domain) и диапазон (range). Фактически домен – это область определения свойства, а диапазон – область значений свойства. Иными словами, свойство связывает индивиды из домена с индивидами из диапазона. Например, в онтологии пицц свойство *имеетНачинку* должно связывать индивиды из класса *Пицца* с индивидами из класса *Начинка*. Таким образом, доменом свойства *имеетНачинку* является класс *Пицца*, а диапазоном – класс *ПиццаНачинка*.

Очень важно понимать, что домен и диапазон свойства в OWL не рассматриваются как строгие ограничения на индивиды, которые связываются этим свойством. Например, с помощью свойства *имеетНачинку* можно связать индивид класса *Пирог* с индивидом класса *ПирогНачинка*.

Однако домены и диапазоны используются машиной вывода как аксиомы. Если A связан с индивидом B свойством P (т.е. существует утверждение $A P B$) с доменом C и рангом D , то машина логического вывода сделает заключение, что индивид A является членом класса C , а индивид B является членом класса D . Например, из того факта, что индивид класса *Пирог* связан с индивидом класса *ПирогНачинка* свойством *имеетНачинку* (с доменом *Пицца*), машина логического вывода сделает заключение, что класс *Пирог* – это подкласс класса *Пицца*, а *ПирогНачинка* это подкласс класса *ПиццаНачинка*. Очевидно, что этот вывод неверен. Если же пары классов *Пицца* и *Пирог*, *ПирогНачинка* и *ПиццаНачинка* описаны как непересекающиеся, то машина вывода заключит, что онтология противоречива.

Таким образом, использование доменов и диапазонов при описании свойств может привести к различным проблемам и непредсказуемым результатам при логическом выводе. В больших онтологиях найти причину подобных проблем бывает очень сложно. Поэтому в больших онтологиях не рекомендуется указывать домены и диапазоны свойств. Онтологию пицц, разрабатываемую в данном пособии, можно считать малой, следовательно, вполне допустимо указывать домен и диапазон для свойств.

Для того чтобы добавить домен и диапазон, используют кнопку  (Добавить) рядом с **Domain(intersection)** и **Range(intersection)** соответственно в области окна **Description** (рис. 4.4).



Рисунок 4.4. Область Description вкладки Object Properties

Упражнение 10. Добавление домена и диапазона для свойств *имеетНачинку*, *имеетОснову*

1. В иерархии свойств выберите свойство *имеетНачинку*.
2. Нажмите кнопку  (Добавить) рядом с **Range(intersection)** в области окна **Description**.
3. В появившемся окне выберите вкладку **Class hierarchy**, из иерархии классов выберите класс *ПиццаНачинка* и нажмите **ОК**.
4. Нажмите кнопку  (Добавить) рядом с **Domain (intersection)** в области окна **Description**.

5. В появившемся окне выберите вкладку **Class hierarchy**, из иерархии классов выберите класс *Пицца* и нажмите **ОК**.
6. В иерархии свойств выберите свойство *имеетОснову*.
7. Повторите шаги 2-5, задав в качестве диапазона свойства – класс *ПиццаОснова*, а в качестве домена – класс *Пицца*.

В общем случае домен и диапазон свойства являются диапазоном и доменом соответственно для обратного ему свойства. Например, если для свойства *имеетНачинку* имеем домен *Пицца* и диапазон *ПиццаНачинка*, то обратное ему свойство *являетсяНачинкой* должно иметь домен *ПиццаНачинка* и диапазон *Пицца*.

Упражнение 11. Добавление домена и диапазона для свойств являетсяНачинкой, являетсяОсновой

1. Выполните данное упражнение самостоятельно.

Заметим, что для одного свойства в Protégé можно указать более одного домена и диапазона. Это означает, что доменом и диапазоном этого свойства будет пересечение указанных классов. Например, доменом свойства *имеетДекретныйОтпуск* может быть пересечение классов *Женицина* и *Сотрудник*.

4.4. Функциональные свойства

Свойство *P* называется функциональным, если для любого индивида *A* существует не более чем один индивид *B*, с которым индивид *A* связан свойством *P*. Иными словами, если свойство функциональное, то любой индивид может быть связан этим свойством только с одним другим индивидом.

Например, рассмотрим свойство *имеетМать*. Очевидно, что это функциональное свойство; так, у любого человека может быть только одна мать.

Пусть в некоторой онтологии индивид *Иван* связан свойством *имеетМать* с двумя индивидами *Ольга* и *Оля*. Если в рассматриваемой онтологии явно указано, что свойство *имеетМать* функциональное, то будет сделан вывод, что *Ольга* и *Оля* – один и тот же индивид. Если же в этой онтологии явно указано, что *Ольга* и *Оля* – различные индивиды, то утверждения *Иван имеетМать Ольга* и *Иван имеетМать Оля* будут противоречивыми (т.е. нарушают целостность данных).

Функциональные свойства также называют свойствами с одним значением или характеристиками (features).

Обратное функциональное свойство – это обратное свойство, являющееся функциональным.

Свойство P называется обратно-функциональным свойством, если для любого индивида B существует не более чем один индивид A , который связан с индивидом B свойством P . Иными словами, если свойство обратно-функциональное, то может существовать только один индивид, который связан этим свойством с любым другим индивидом.

Примером обратно-функционального свойства в рассмотренном выше примере может быть свойство *являетсяМатерью*. Если мы имеем два утверждения *Ольга являетсяМатерью Ивана* и *Оля являетсяМатерью Ивана*, и свойство *являетсяМатерью* объявлено как обратно-функциональное то это означает, что *Ольга* и *Оля* – один и тот же индивид, в противном случае онтология противоречива.

В онтологии пицц свойство *имеетОснову* является функциональным, так как конкретная пицца может иметь только одну основу.

Упражнение 12. Описание свойства имеетОснову как функционального.

1. В иерархии свойств выберите свойство *имеетОснову*.
2. В области окна **Characteristics** отметьте характеристику **Functional (Функциональное)**.

4.5. Транзитивные свойства

Свойство P называется транзитивным, если из фактов, что свойство P связывает индивиды A и B и свойство P связывает индивиды B и C , следует, что свойство P связывает индивиды A и C .

Рассмотрим пример транзитивного свойства *имеетПотомка*. Предположим, в некоторой онтологии индивид *Ольга* связан свойством *имеетПотомка* с индивидом *Иван*, индивид *Иван* связан свойством *имеетПотомка* с индивидом *Михаил*. Если свойство *имеетПотомка* описано в данной онтологии как транзитивное, то из этих двух

утверждений следует, что индивид *Ольга* связан свойством *имеетПотомка* и с индивидом *Михаил*.

В онтологии пицц свойство *имеетИнгредиент* является транзитивным. Очевидно, что если начинка пиццы имеет некоторые ингредиенты, то и сама пицца имеет эти ингредиенты. Другими словами, если индивид *A* класса *Пицца* связан свойством *имеетИнгредиент* с индивидом *B* класса *ПиццаНачинка*, а индивид *B* связан свойством *имеетИнгредиент* с некоторым индивидом *C*, то справедливо утверждение «пицца *A* имеетИнгредиент *C*».

Отметим, что свойство, обратное транзитивному, также является транзитивным. Таким образом, если свойство *имеетИнгредиент* является транзитивным, то и свойство *являетсяИнгредиентом* тоже является транзитивным.

Упражнение 13. Описание свойств *имеетИнгредиент* и *являетсяИнгредиентом* как транзитивных

1. В иерархии свойств выберите свойство *имеетИнгредиент*.
2. В области окна **Characteristics** отметьте характеристику **Transitive (Транзитивное)**.
3. Повторите шаги 1-2 для свойства *являетсяНачинкой*.

Кроме того, если свойство транзитивное, оно не может быть функциональным (это просто не имеет смысла).

4.6. Симметричные свойства

Свойство *P* называется симметричным, если из факта, что свойство *P* связывает индивиды *A* и *B*, следует, что свойство *P* связывает индивиды *B* и *A*.

Примером симметричного свойства может служить свойство *являетсяСестрой*. Действительно, если справедливо утверждение «*Ольга являетсяСестрой Татьяны*», то справедливо и утверждение «*Татьяна являетсяСестрой Ольги*». Ещё одним примером симметричного свойства может быть свойство «*сравнимПоПродажамС*»: если справедливо утверждение «*Мерседес-Бенц сравнимПоПродажамС Ауди*», то справедливо и утверждение «*Ауди сравнимПоПродажамС Мерседес-Бенц*».

4.7. Асимметричные свойства

Свойство P называется асимметричным, если из факта, что свойство P связывает индивиды A и B , не может следовать, что свойство P связывает индивиды B и A .

Примером асимметричного свойства может служить свойство *являетсяМатерью*. Действительно, если справедливо утверждение «*Ольга являетсяМатерью Елены*», то утверждение «*Елена являетсяМатерью Ольги*» не может быть справедливым. Ещё одним примером асимметричного свойства может быть свойство «*превосходитПоПродажам*»: если справедливо утверждение «*Ауди превосходитПоПродажам Сеат*», то утверждение «*Сеат превосходитПоПродажам Audi*» не может быть справедливым.

4.8. Рефлексивные свойства

Свойство P называется рефлексивным, если оно связывает любой индивид A сам с собой.

Примером рефлексивного свойства может служить свойство *знает*. Очевидно, что любой индивид класса *Человек* связан этим свойством сам с собой. Например, справедливо утверждение «*Ольга знает Ольга*». Из этого, однако, не следует, что индивид *Ольга* не связан свойством *знает* с другим индивидами класса *Человек*.

4.9. Иррефлексивные свойства

Свойство P называется иррефлексивным, если оно не может связывать любой индивид A сам с собой.

Примером иррефлексивного свойства может служить свойство *являетсяМатерью*. Очевидно, что любой индивид класса *Человек* не может быть связан этим свойством сам с собой. Например, утверждение «*Ольга являетсяМатерью Ольги*» не может быть справедливым, если *Ольга* – один и тот же индивид.

Упражнение 14. Определение характеристик свойств онтологии Пицца

1. Для каждого свойства онтологии *Пицца* самостоятельно проанализируйте, какими характеристиками оно обладает, и укажите эти характеристики.

Обратите внимание, если некоторая характеристика свойства явно не указана, нельзя сделать вывод о её наличии или отсутствии. Например, если не указано, что свойство *являетсяМатерью* является асимметричным, то машина логического вывода из утверждения «*Ольга являетсяМатерью Елена*» не сможет сделать вывод о справедливости или несправедливости утверждения «*Елена являетсяМатерью Ольги*». Однако если вы явно сделаете утверждение об асимметричности свойства, то машина вывода признает утверждение «*Елена являетсяМатерью Ольги*» противоречивым.

Глава 5. ОПИСАНИЕ И ОПРЕДЕЛЕНИЕ КЛАССОВ

На основе имеющихся в онтологии свойств можно описывать и определять новые классы.

5.1. Ограничения

Напомним, что в языке OWL свойства описывают бинарные отношения. Свойства типа данных описывают отношения между индивидами и литеральными значениями. Объектные свойства описывают отношения между двумя индивидами. В OWL можно создавать классы, используя ограничения на свойства (аксиомы классов с ограничениями на свойства).

Ограничение (restriction) описывает класс на основе отношений, в которых участвуют индивиды этого класса. Можно сказать, что ограничение и именованный класс – это различные виды классов.

Приведём примеры ограничений.

Класс индивидов, которые связаны свойством *имеетРебенка* хотя бы с одним другим индивидом, т.е. по сути, класс, описывающий родителей.

Класс индивидов, которые связаны свойством *имеетРебенка*, по крайней мере, с тремя другими индивидами, т.е. по сути, класс, описывающий многодетных родителей.

Класс индивидов, которые связаны свойством *имеетРебенка* хотя бы с одним другим индивидом класса *Женщины*, т.е. по сути, класс, описывающий родителей, имеющих дочерей.

Класс индивидов, которые связаны свойством *имеетНачинку*, по крайней мере, с одним индивидом класса *МоцареллаНачинка*, т.е. класс индивидов, которые имеют, по крайней мере, один вид начинки из моцареллы.

Класс индивидов, которые связаны свойством *имеетНачинку* только с индивидом класса *ОвощнаяНачинка*, т.е. по сути, класс индивидов, имеющих только овощную начинку.

В OWL все приведённые выше классы можно описать с помощью ограничений (аксиом классов с ограничениями на свойства). OWL-ограничения делятся на три категории:

- кванторные ограничения (Quantifier Restrictions),
- ограничения кардинальности (Cardinality Restrictions),
- ограничения на значения (hasValue Restrictions).

Прежде всего, рассмотрим кванторные ограничения, которые, в свою очередь, можно разделить на ограничения существования (existential restrictions) и универсальные ограничения (universal restrictions). Ограничения кардинальности будут рассмотрены в пункте 5.8, а ограничения на значения – в главе 6.

5.2. Ограничения существования

Ограничение существования описывают класс индивидов, которые связаны заданным свойством, по крайней мере, с одним индивидом заданного класса.

Например, класс индивидов, которые связаны свойством *имеетНачинку*, по крайней мере, с одним индивидом класса *МоцареллаНачинка*, т.е. класс индивидов, которые имеют по крайней мере один вид начинки из моцареллы. В Protégé для обозначения ограничений существования используется ключевое слово **some**. Таким образом, приведённое выше ограничение будет записано в Protégé следующим образом *имеетНачинку some МоцареллаНачинка*.

Ограничения существования представляют собой наиболее общий тип ограничений в OWL-онтологиях. Часто для записи этого ограничения в других инструментах онтологического моделирования используют квантор существования $\exists$, т.е. приведённое выше ограничение может быть записано следующим образом: $\exists$ *имеетНачинку МоцареллаНачинка*. Данная запись соответствует формальному синтаксису дескрипционных логик.

Ограничения описывают так называемые **анонимные**, т.е. безымянные классы. Анонимный класс содержит все индивиды, удовлетворяющие ограничению, т.е. все индивиды, которые связаны заданными свойствами с индивидами заданных классов по заданным правилам.

Ограничения для классов создаются и редактируются с помощью окна **Description**, на вкладке **Classes**, изображённой на рис. 5.1. Именно это окно отображает всю информацию о выделенном классе. Ограничения, которые используются в описании OWL классов, задаются в этом окне в секции **SubClass Of**.

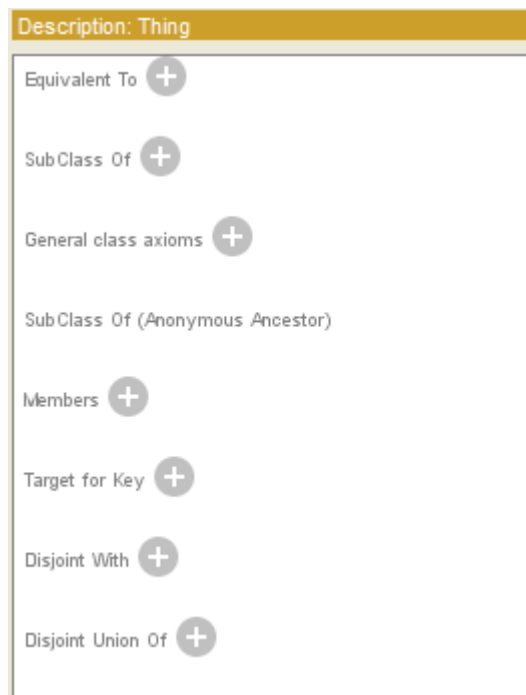


Рисунок 5.1. Class Description View

Для того чтобы создать ограничение на класс нужно выделить выделить этот класс в иерархии классов и затем нажать на кнопку  (Добавить) в нужной секции окна **Description**.

Упражнение 15. Добавление ограничения: каждая пицца имеет основу

1. Выберите класс *Пицца* в иерархии классов.
7. Нажмите кнопку  (Добавить) рядом с **SubClass Of** в окне области окна **Description:Пицца** (рис 5.2).
2. В появившемся окне выберите вкладку **Object restriction creator (Мастер создания объектного ограничения)** (рис. 4.3)
3. В левой части окна (**Restricted property – Ограничиваемое свойство**) из иерархии свойств выберите свойство *имеетОснову*.
4. В правой части окна (**Restriction filler – Фильтр ограничения**) из иерархии классов выберите класс *ПиццаОснова*.
5. Убедитесь, что в нижней части окна в области **Restriction type (Тип ограничения)** выбрано ограничение **Some** и нажмите **OK**.

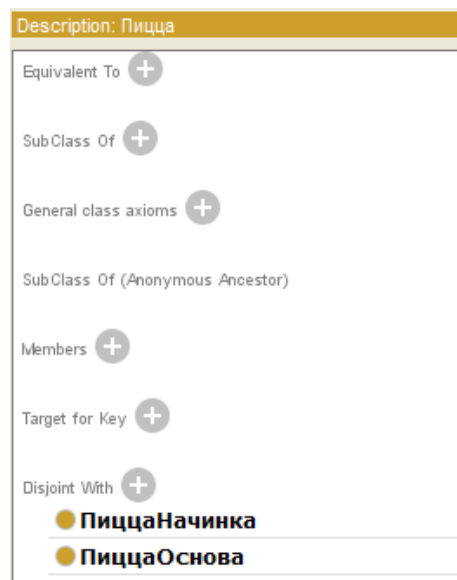


Рисунок 5.2. Область окна Description:Пицца

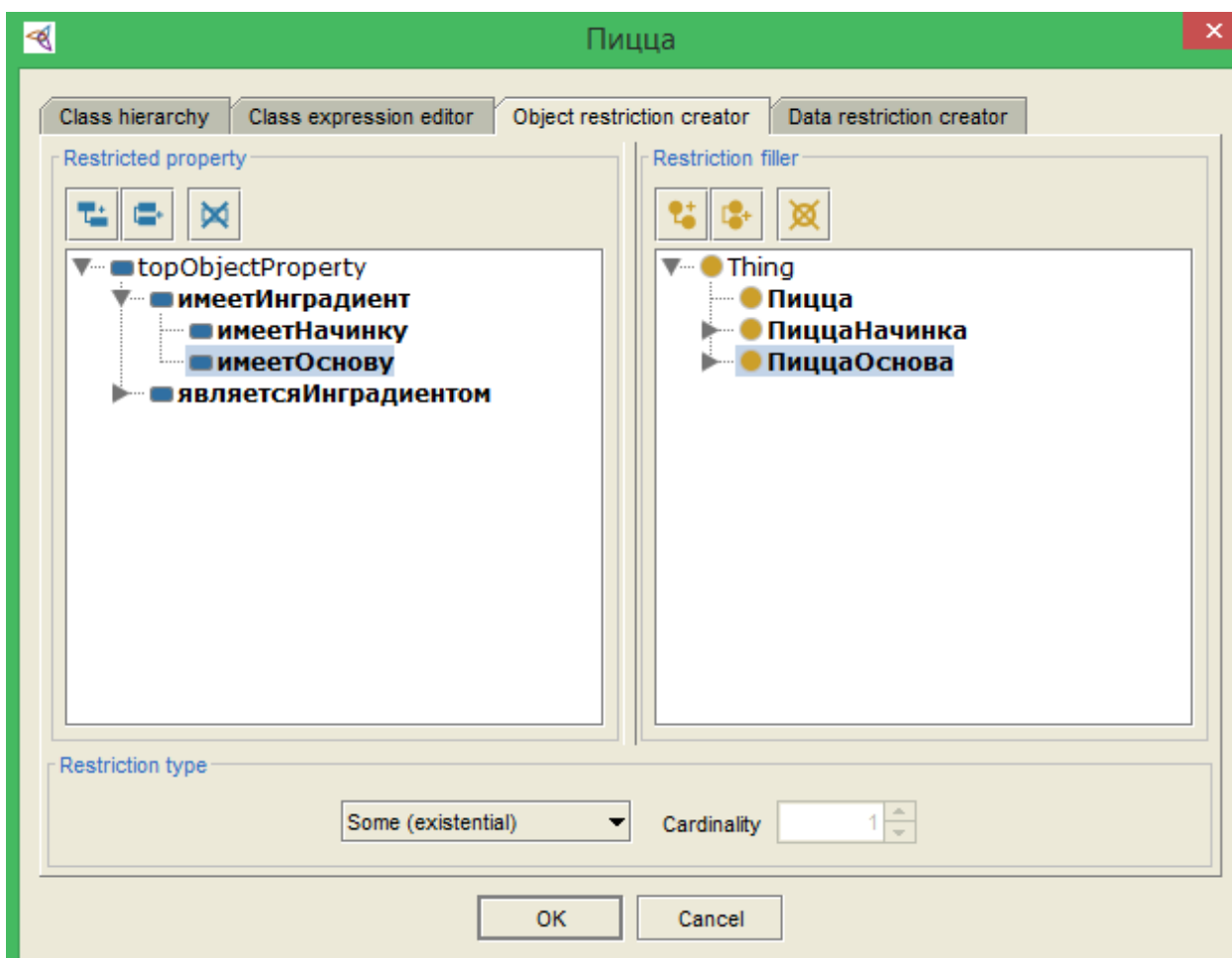


Рисунок 5.3. Создание ограничения на объектное свойство

Аналогичные действия (замена шагов 2-5 в упражнении 15) можно выполнить с помощью вкладки **Class Expression Editor (редактор выражений)**, где можно непосредственно напечатать ограничение:

имеетОснову some ПиццаОснова. При этом удобно использовать комбинацию клавиш **Ctrl+Пробел** или клавишу **Alt** для автоподстановки имён классов и свойств по их начальным буквам.

Таким образом, мы описали класс *Пицца* как подкласс класса *Thing* и подкласс анонимного класса индивидов, имеющих, по крайней мере, одну начинку.

Заметим, что эти два условия являются **необходимыми условиями** для принадлежности индивида к классу.

Упражнение 16. Создание классов ИменованнаяПицца и МаргаритаПицца

1. Выберите класс *Пицца* в иерархии классов.
2. Создайте подкласс класса *Пицца* – *ИменованнаяПицца*.
3. Создайте подкласс класса *ИменованнаяПицца* – *МаргаритаПицца*.
4. Добавьте комментарий к классу *МаргаритаПицца*: «Пицца, имеющая начинку из моцареллы и помидоров».

Заметим, что документирование онтологии (в частности, создание аннотаций к классам и свойствам) является хорошей практикой, так как позволяет улучшить взаимодействие между различными разработчиками онтологии.

Упражнение 17. Создание ограничения существования на класс МоцареллаПицца

1. Убедитесь, что в иерархии классов выбран класс *МаргаритаПицца*.
2. Нажмите кнопку  (Добавить) рядом с **SubClass Of** в окне области окна **Description:МаргаритаПицца**.
3. Создайте ограничение *имеетНачинку some МоцареллаНачинка*.
4. Повторите шаг 4 и затем создайте ограничение *имеетНачинку some МоцареллаНачинка*.
5. Убедитесь, что окно **Description:МаргаритаПицца** выглядит, как показано на рис. 5.4.

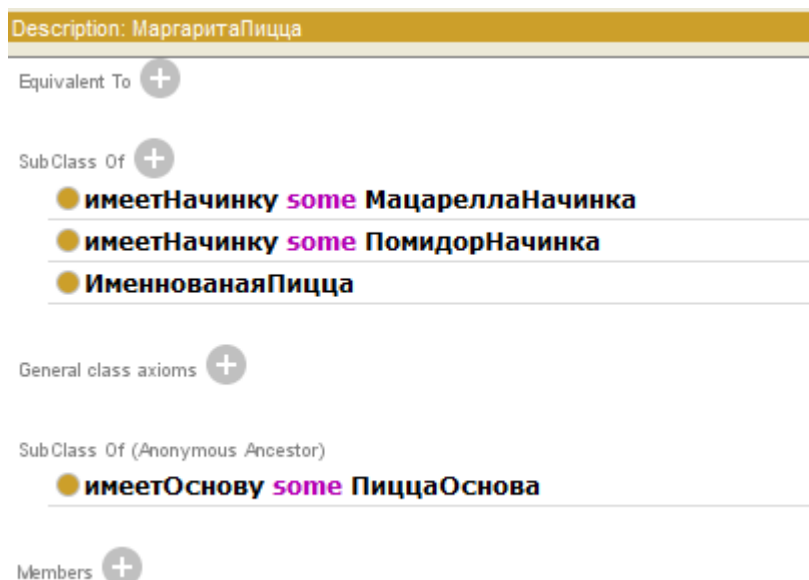


Рисунок 5.4. Окно Description:МаргаритаПицца

Таким образом, для того чтобы некоторый индивид был членом класса *МаргаритаПицца*, необходимо выполнение двух условий: чтобы он был членом класса *ИменованнаяПицца* и членом анонимного класса, который связан свойством *имеетНачинку* по крайней мере, с одним индивидом класса *МоцареллаНачинка* и, по крайней мере, одним индивидом класса *ПомидорНачинка*.

Создадим класс, описывающий пиццу Американо, которая имеет начинку из пепперони, моцареллы и помидоров. Можно заметить, что данный класс отличается от класса *МаргаритаПицца* только одним ограничением: *имеетНачинку some ПепперониНачинка*, потому целесообразно создавать данный класс так называемым *клонированием*.

Упражнение 18. Создание класса АмериканоПицца

1. Выберите класс *МаргаритаПицца* в иерархии классов.
2. Выберите в меню **Edit** пункт **Duplicate selected class (Клонировать выбранный класс)**.
3. Заполните появившееся окно так, как показано на рисунке 5.5, и нажмите ОК.
4. Измените комментарий к классу *АмериканоПицца*: «Пицца, имеющая начинку из моцареллы, помидоров и пепперони».
5. Создайте ограничение *имеетНачинку some ПепперониНачинка*.
6. Убедитесь, что описание класса выглядит, как показано на рис. 5.6.

Duplicate Class

Name:

IRI:

Where you would like to duplicate the class?

☒ active ontology

☐ original ontology(ies)

☒ Duplicate annotations

Рисунок 5.5. Создание клона класса

Description: АмериканоПицца

Equivalent To

SubClass Of

- имеетНачинку some МацареллаНачинка
- имеетНачинку some ПепперониНачинка
- имеетНачинку some ПомидорНачинка
- ИменованнаяПицца

General class axioms

SubClass Of (Anonymous Ancestor)

- имеетОснову some ПиццаОснова

Рисунок 5.6. Окно Description:АмериканоПицца

Упражнение 19. Создание класса АмериканоХотПицца и СохоПицца

1. Создайте самостоятельно класс *АмериканоХотПицца*, описывающий именованные пиццы, которые имеют такую же начинку, как пицца *Американо* и, кроме того, начинку из перца халапеньо.
2. Убедитесь, что описание класса выглядит, как показано на рис. 5.7.

3. Создайте самостоятельно класс *СохоПицца*, описывающий именованные пиццы, который имеют такую же начинку, как пицца Маргарита и, кроме того, начинку из оливок и пармезана.
4. Убедитесь, что описание класса выглядит, как показано на рис. 5.8.

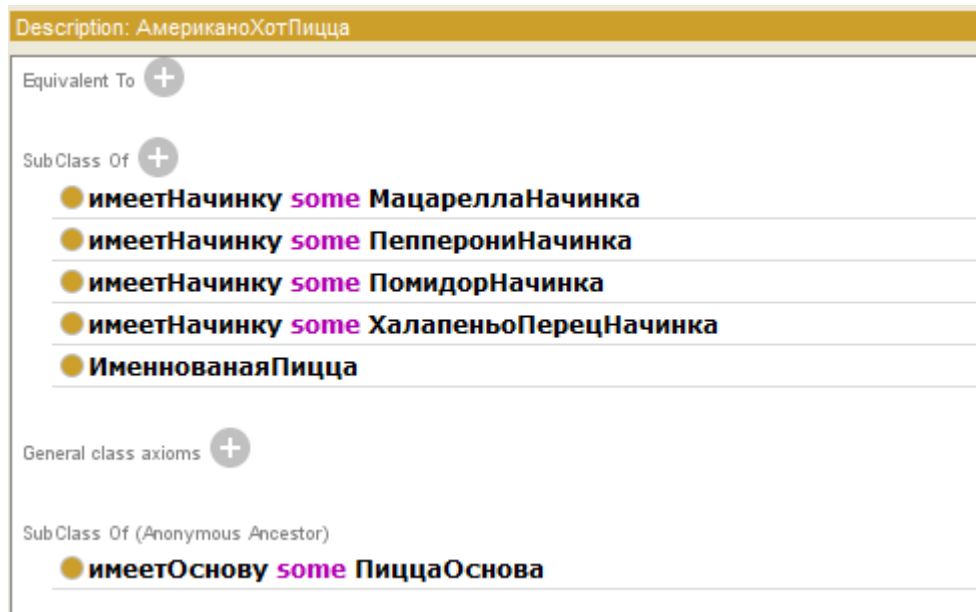


Рисунок 5.7. Окно Description:АмериканоХотПицца

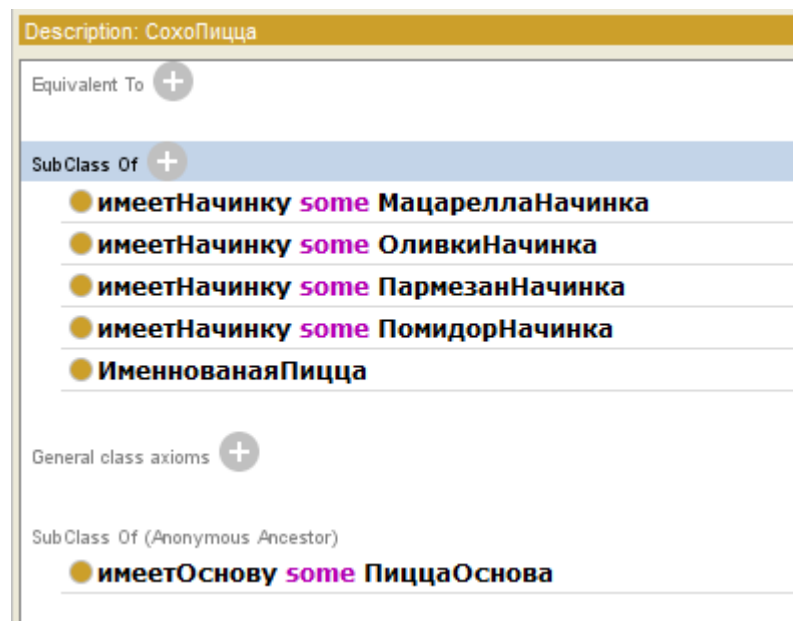


Рисунок 5.8. Окно Description:СохоПицца

Упражнение 20. Объявление подклассов класса ИменованнаяПицца непересекающимися

1. Выберите класс *МаргаритаПицца* в иерархии классов.
2. В меню **Edit** выберите пункт **Make primitive sibling disjoint**.

5.3. Использование машины логического вывода

Любая онтология, созданная на языке OWL DL¹, может быть обработана логической машиной вывода (reasoner). В частности, машина вывода может достраивать иерархию классов, выявляя неописанные отношения вида «класс-подкласс». Такого рода проверка называется также классификационным тестом. В результате работы машины логического вывода строится так называемая **выведенная (inferred) иерархия классов**. Из-за этой функции машины логического вывода также называют классификаторами, что не совсем корректно, так как они выполняют и другую функцию.

Кроме осуществления классификации, машина вывода может проверять онтологию на согласованность (непротиворечивость). Основываясь на описании класса, машина вывода заключает, может ли данный класс содержать какие-либо индивиды. Если это невозможно, то класс считается несогласованным (противоречивым). Кроме того, если онтология наполнена индивидами, то, основываясь на описании этих индивидов и классов, машина вывода может отнести тот или иной индивид к какому-либо классу или заключить, что описание индивидов класса противоречит описанию класса.

Использовать машину логического вывода рекомендуется регулярно на двух этапах разработки онтологии: при добавлении любых аксиом (характеристик свойств, ограничений); при наполнении онтологии индивидами.

Насколько регулярно использовать машину логического вывода (например, при добавлении одной или нескольких аксиом), решает сам разработчик. Так как предполагается, что для читателя данного пособия это первый опыт разработки, рекомендуется использовать далее машину логического вывода после внесения любых изменений указанного типа. (Несмотря на наличие в машине вывода системы объяснений, неопытному разработчику иногда бывает трудно найти причину ошибок, возникших после внесения целого ряда изменений).

Для того чтобы в Protégé использовать машину логического вывода, необходимо в меню **Reasoner** выбрать одну из предлагаемых машин вывода (например, **Hermit 1.3.8.3** или **Fact++**) и затем выбрать пункт **Start reasoner (Запустить машину логического вывода)**.

¹ OWL DL – один из двух основных диалектов языка OWL. Он основан на использовании аппарата дескрипционных логик, что обеспечивает эффективный логический вывод, т.е. вывод, который гарантированно будет завершен за конечное время.

Для демонстрации возможностей машины вывода по выявлению противоречивых классов создадим класс, который является одновременно подклассом классов *СырНачинка* и *ОвощиНачинка*. Такая стратегия часто используется для проверки корректности построенной онтологии. Классы, которые добавляются в онтологию с целью подобных проверок, часто называют пробными классами.

Упражнение 21. Добавление в онтологию пробного класса

1. Выберите класс *СырНачинка* и иерархии классов.
2. Создайте подкласс класса *СырНачинка* – *ПробныйКласс*.
3. Убедитесь, что выделен класс *ПробныйКласс*.
4. В окне **Description:ПробныйКласс** в области **Subclass of** добавьте класс *ОвощиНачинка*.
5. Убедитесь, что описание класса выглядит, как показано на рис. 5.9.

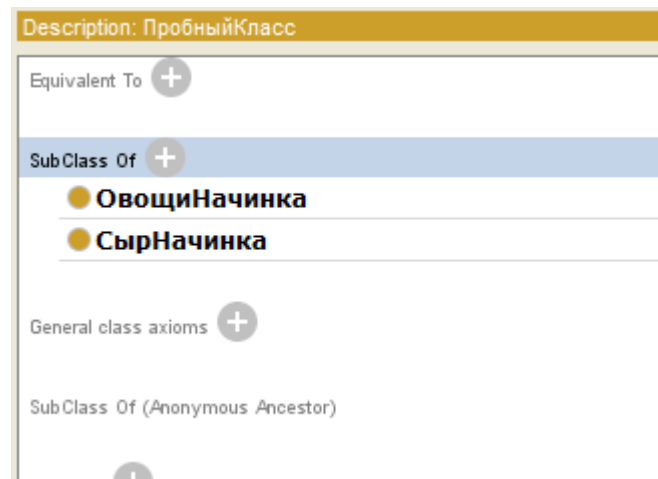


Рисунок 5.9. Окно Description:ПробныйКласс

Упражнение 22. Использование машины вывода для выявления противоречивости класса ПробныйКласс

1. В меню **Reasoner** выберите одну из предлагаемых машин вывода (например, **Hermit 1.3.8.3**) и затем выберите пункт **Start reasoner**.
2. Проанализируйте получившиеся результаты, а именно обратите внимание, что в иерархии классов *ПробныйКласс* изображён два раза (как подкласс класса *ОвощиНачинка* и как подкласс класса *СырНачинка*) и выделен красным цветом (рис. 5.10).
3. Перейдите на вкладку **Class hierarchy (inferred)** и проанализируйте выведенную иерархию классов (рис. 4.11), а именно обратите внимание, что *ПробныйКласс*, является подклассом класса *Nothing*

(т.е. класса для которого не может существовать индивидов) и таким образом является противоречивым.

4. В окне **Description: Пробный класс** рядом с **Nothing** нажмите кнопку  (**Explain inference – объяснить вывод**).
5. В появившемся окне (рис. 4.12) проанализируйте объяснение противоречивости класса. Суть данного объяснения заключается в том, что *ПробныйКласс* является подклассом классов *ОвощиНачинка* и *СырНачинка*, а эти классы являются непересекающимися, т.е. *ПробныйКласс* не может содержать индивидов.

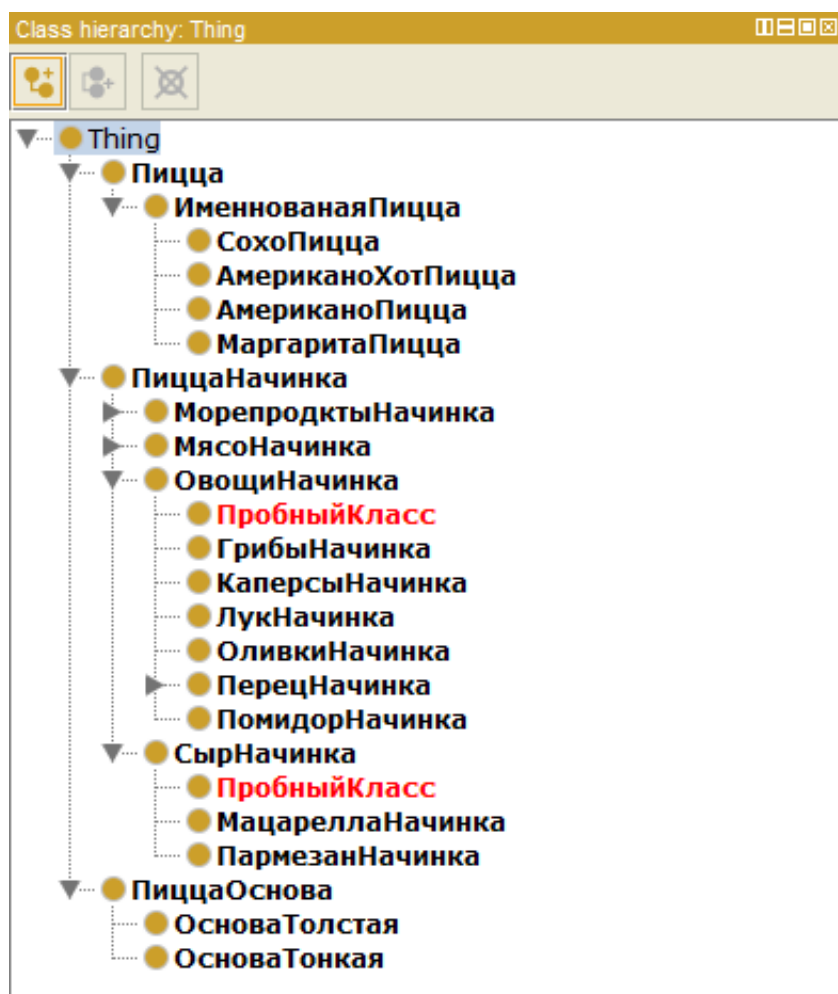


Рисунок 5.10. Иерархия классов

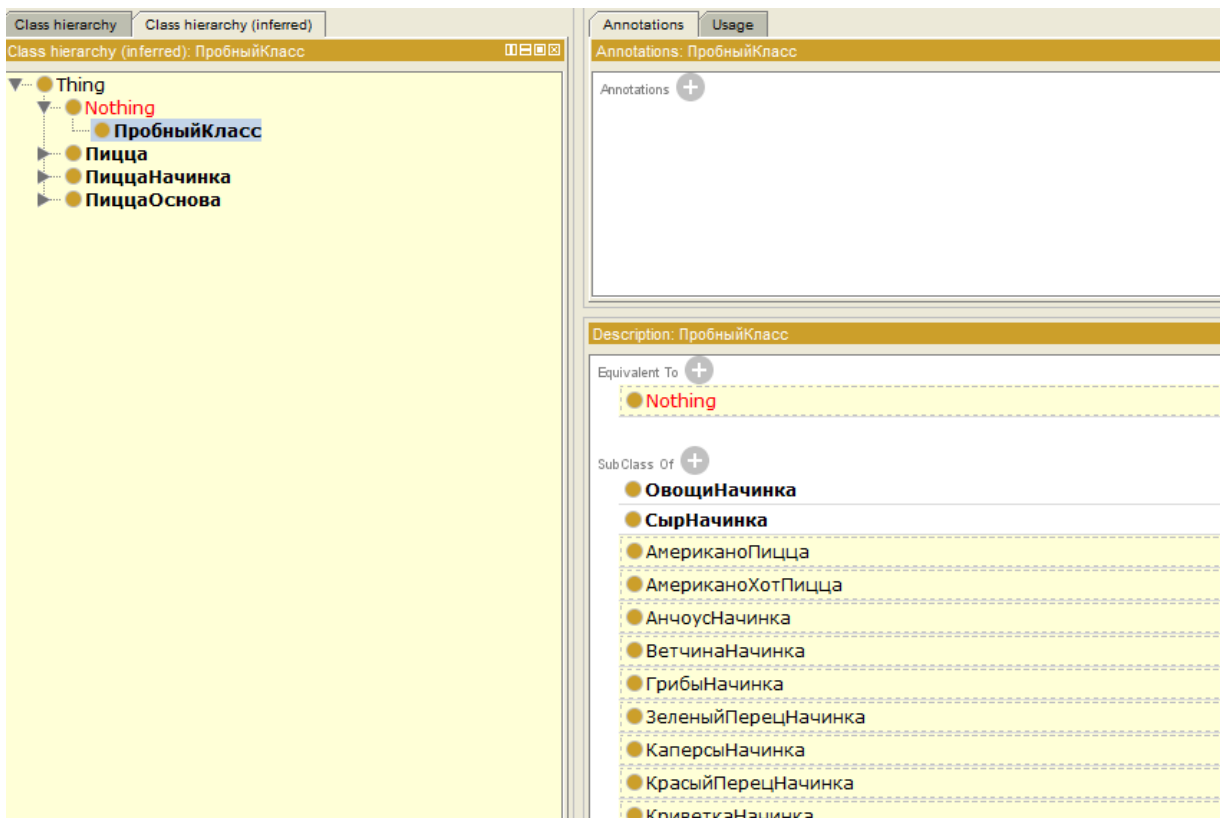


Рисунок 5.11. Выведенная иерархия классов



Рисунок 5.12. Объяснение противоречивости класса ПробныйКласс

Упражнение 23. Удаление свойства непересекаемости между классами ОвощиНачинка и СырНачинка

1. Выделите класс *СырНачинка* в иерархии классов.
2. Удалите аксиому о непересекающихся классах.
3. Запустите машину логического вывода.
4. Проанализируйте результаты: убедитесь, что класс *ПробныйКласс* является непротиворечивым.
5. Удалите класс *ПробныйКласс*.
6. Сделайте все подклассы класса *ПиццаНачинка* непересекающимися.

5.4. Необходимые и достаточные условия

Все созданные в предыдущих разделах классы описаны только так называемыми необходимыми условиями, т.е. для того, чтобы индивид был членом описанного класса необходимо, чтобы для него выполнялись эти условия. Необходимое условие не позволяет утверждать, что если для некоторого индивида выполняется это условие, то он должен являться членом этого класса.

Класс, описанный только с помощью необходимых условий, называется **примитивным классом (primitive class)**.

Рассмотрим пример. Создадим подкласс класса *Пицца* – *СырнаяПицца*, т.е. класс, описывающий пиццы хотя бы с одним видом сырной начинки. Причём данное условие запишем как ограничение существования, т.е. как необходимое условие.

Упражнение 24. Создание класса *СырнаяПицца*

1. Выделите класс *Пицца* в иерархии классов.
2. Создайте подкласс *СырнаяПицца*.
3. Добавьте ограничение существования *имеетНачинка some СырНачинка*.

Рассмотрим некоторый индивид А. Предположим, что известно, что он является членом класса *Пицца* и при этом имеет хотя бы одну начинку из сыра. Однако текущего описания класса *СырнаяПицца* *недостаточно*, чтобы сделать вывод что индивид А является членом класса *СырнаяПицца*. Для того чтобы это утверждать, надо описать класс *СырнаяПицца* с помощью «необходимого и достаточного условия». Т.е. условие, что «*СырнаяПицца* является подклассом класса *Пицца* и *имеетНачинка some СырНачинка*», должно являться *необходимым и достаточным условием*.

Класс, описанный с помощью необходимых и достаточных условий, называется **определённым классом (defined class)**. Условие, являющееся необходимым и достаточным, в Protégé называется **эквивалентным классом**.

Упражнение 25. Конвертирование необходимого условия для класса *СырнаяПицца* в необходимое и достаточное.

1. Выделите класс *Пицца* в иерархии классов.

2. В меню **Edit** выберите пункт **Convert to defined class** (Конвертировать в определенный класс).
3. Убедитесь, что описание класса выглядит, как показано на рис. 5.13.

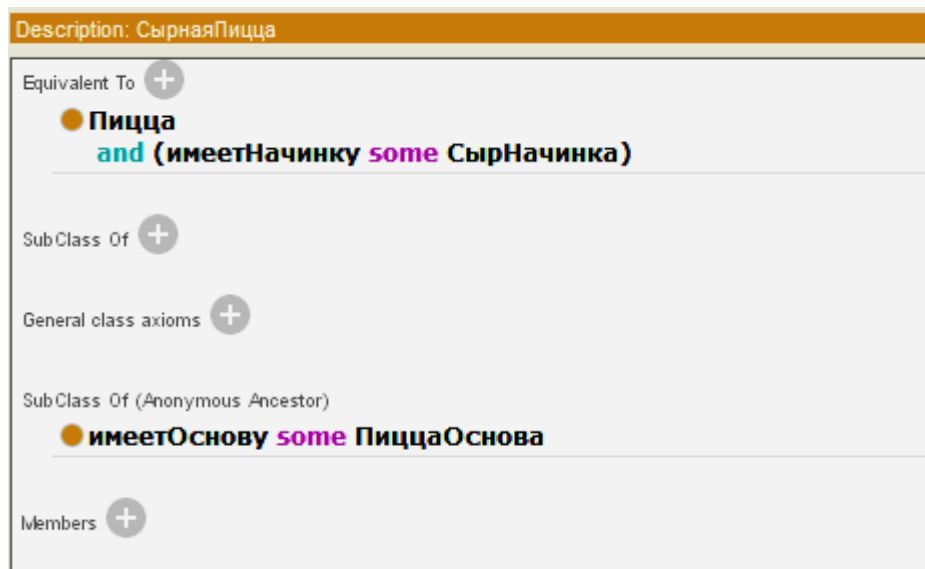


Рисунок 5.13. Окно Description: СырнаяПицца

Созданное необходимое и достаточное условие позволит сделать вывод, что индивид A , описанный выше, является членом класса *СырнаяПицца*.

Обобщим вышесказанное.

Если класс A описан с помощью **необходимых условий**, можно сделать вывод, что каждый индивид этого класса должен удовлетворять этим условиям, но нельзя сделать вывода, что если некоторый индивид удовлетворяет этим условиям, то он является членом класса A .

Если класс A описан с помощью **необходимых и достаточных условий**, можно сделать вывод, что каждый индивид этого класса должен удовлетворять этим условиям, и что если некоторый индивид удовлетворяет этим условиям, то он является членом класса A .

В OWL возможно создать множественный набор необходимых и достаточных условий.

Обратите внимание, что примитивные классы обозначены в Protégé круглыми жёлтыми ярлыками, а определённые – в Protégé круглыми жёлтыми ярлыками с тремя горизонтальными белыми линиями (смотри класс *СырнаяПицца*).

При разработке онтологии вручную для каждого класса указывается только один надкласс, а затем на основе необходимых и достаточных условий машина вывода перестроит иерархию классов.

Упражнение 26. Использование машины вывода для определения подклассов класса СырнаяПицца

1. Запустите машину вывода.
2. Проанализируйте выведенную онтологию (рис. 5.14), а именно обратите внимание, что классы *АмериканоПицца*, *АмериканоХотПицца*, *МаргаритаПицца*, *СохоПицца* автоматически определились как подклассы класса *СырнаяПицца*, так как каждый из этих видов пиццы удовлетворяет необходимым и достаточным условиям в описании класса *СырнаяПицца*, а именно являются подклассами класса *Пицца* и имеют хотя бы одну сырную начинку.
3. Обратите внимание, что классы *АмериканоПицца*, *АмериканоХотПицца*, *МаргаритаПицца*, *СохоПицца* в выведенной онтологии имеют два надкласса: *ИменованнаяПицца* и *СырнаяПицца*.

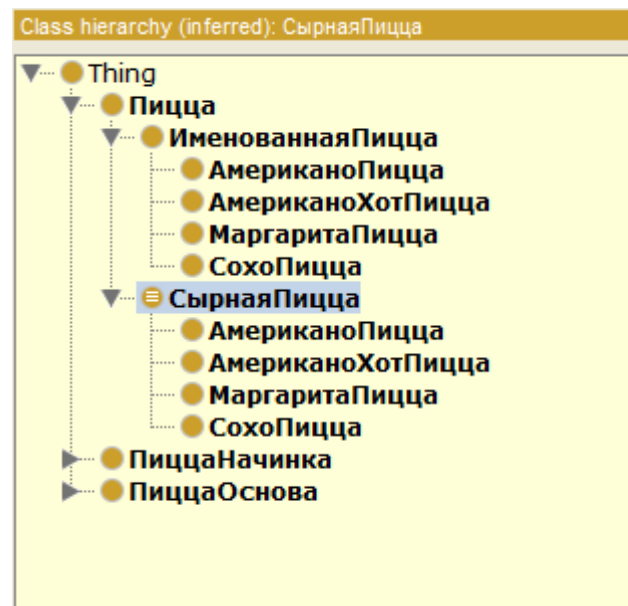


Рисунок 5.14. Выведенная онтология пиццы

Отметим, что в общем случае машина вывода может перестроить отношения «класс-подкласс» только для определённых классов. Исключением является случай, если в качестве домена свойства выступает примитивный класс.

5.5. Универсальные ограничения

Как было отмечено выше, все кванторные ограничения делятся на ограничения существования и универсальные ограничения.

Все рассмотренные выше ограничения являлись ограничениями существования. Они описывают класс индивидов, которые связаны заданным свойством, по крайней мере, с одним индивидом заданного класса. Однако ограничения существования не позволяют выразить тот факт, что все индивиды описываемого класса связаны заданным свойством только с индивидами заданного класса.

Например, мы использовали ограничение *имеетНачинку some ОвощнаяНачинка*. Оно означает, что все индивиды описываемого класса имеют овощную начинку, но не означает, что все индивиды описываемого класса имеют только овощную начинку.

В этом разделе рассмотрим универсальные ограничения.

Универсальное ограничение описывает класс индивидов, которые связаны заданным свойством исключительно с индивидами заданного класса.

Например, класс индивидов, которые связаны свойством *имеетНачинку* только с индивидами класса *ОвощнаяНачинка*, т.е. по сути, класс индивидов, имеющих только овощную начинку.

В Protégé для обозначения универсальных ограничений используется ключевое слово **only**. Т.е. приведённое выше ограничение будет записано в Protégé следующим образом: *имеетНачинку only ОвощиНачинка*.

Универсальное ограничение обозначается также символом $\forall$. Например универсальное ограничение *имеетНачинку only ОвощиНачинка* может быть записано как $\forall$ *имеетНачинку ОвощиНачинка*.

Создадим класс *ВегетарианскаяПицца*, который описывает пиццы, имеющие только сырную или овощную начинку.

Упражнение 27. Создание класса *ВегетарианскаяПицца*

1. Создайте класс *ВегетарианскаяПицца* как подкласс класса *Пицца*.
2. Убедитесь, что выделен класс *ВегетарианскаяПицца*.
3. Нажмите кнопку  (Добавить) рядом с **SubClass Of** в окне области окна **Description:ВегетарианскаяПицца**.
4. В появившемся окне выберите вкладку **Object restriction creator**.
5. В левой части окна (**Restricted property**) из иерархии свойств выберите свойство *имеетНачинку*.

6. В правой части окна (**Restriction filler**) из иерархии классов выделите сразу два класса, удерживая клавишу CTRL: *СырНачинка* и *ОвощиНачинка*.
7. Убедитесь, что в нижней части окна в области Restriction type выбрано ограничение **Only**, и нажмите **OK**.
8. Убедитесь, что описание класса *ВегетарианскаяПицца* выглядит, как показано на рис. 4.15.

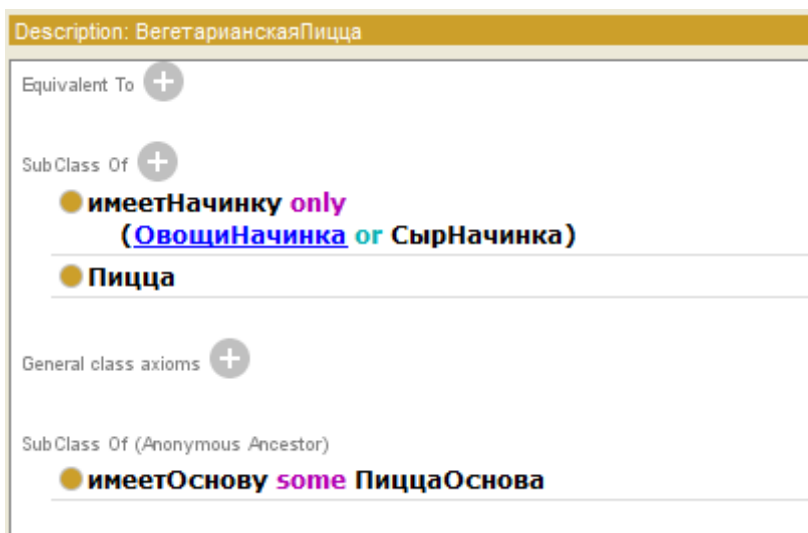


Рисунок 5.15. Окно Description:ВегетарианскаяПицца

Распространённой ошибкой при подобных описаниях является использование в ограничениях на свойство отношения пересечения между классами (*and*) вместо отношения объединения (*or*), в нашем случае: *имеетНачинку only (ОвощиНачинка and СырНачинка)*. Такие ошибки объясняются тем, что на русском языке естественно сказать, что «Вегетарианская пицца – это пицца, которая может иметь только (сырную и овощную начинку)». В нашем примере ограничение *имеетНачинку only (ОвощиНачинка and СырНачинка)* некорректно, так как классы *ОвощиНачинка* и *СырНачинка* являются непересекающимися.

Также некорректно создавать два ограничения: *имеетНачинку only ОвощиНачинка* и *имеетНачинку only СырНачинка*, так как эти два ограничения эквивалентны ограничению *имеетНачинку only (ОвощиНачинка and СырНачинка)*.

Обратите внимание, что универсальное ограничение не означает обязательного существования связи. Например, описание класса *ВегетарианскаяПицца* не позволяет сделать вывод, что данная пицца обязательно имеет начинку, но позволяет сделать вывод, что если

вегетарианская пицца имеет начинки, то эти начинки являются сырными или овощными.

Для того чтобы завершить описание класса *ВегетарианскаяПицца*, нужно необходимое условие его описания преобразовать в необходимое и достаточное условие, т.е. конвертировать класс в определённый класс.

Упражнение 28. Конвертирование класса ВегетарианскаяПицца к определённый класс (выполните самостоятельно).

5.6. Аксиомы замыкания

Используем машину логического вывода, чтобы автоматически достроить иерархию классов. В частности, мы ожидаем, что *МаргаритаПицца* и *СохоПицца* будут определены как подклассы класса *ВегетарианскаяПицца*, так как они удовлетворяют необходимым и достаточным условиям класса: являются пиццами и имеют только сырны и овощные начинки.

Упражнение 29.

1. Запустите машину логического вывода.
2. Убедитесь, что машина вывода не классифицировала классы *МаргаритаПицца* и *СохоПицца* как подклассы класса *ВегетарианскаяПицца*.

Таким образом, работа машины выводов не дала ожидаемых результатов. Класс *ВегетарианскаяПицца* описан корректно, значит ошибка в описании классов *МаргаритаПицца* и *СохоПицца*.

Дело в том, что логический вывод для языка OWL DL основан на так называемом *предположении об открытости мира* (open world assumption, OWA). Такой вывод часто называют выводом открытого мира (open world reasoning, OWR).

Предположение об открытости мира означает, что мы не можем предполагать, что чего-то не существует, пока явно не сказано, что этого не существует. Другими словами, если нет явного утверждения, что что-либо ложно, это не значит, что это ложно.

В случае с онтологией пиццы мы описали класс *МаргаритаПицца*, как класс индивидов, имеющих начинку из моцареллы и помидоров. Однако в соответствии с предположением об открытости мира это не

означает, что пицца Маргарита не имеет других начинок. Поэтому пока мы явно не скажем, что пицца Маргарита имеет начинки исключительно из моцареллы и помидоров, машина вывода не сможет классифицировать этот класс как подкласс класса *ВегетарианскаяПицца*. Для того чтобы сказать, что пицца Маргарита имеет начинки исключительно из моцареллы и помидоров, необходимо наложить ограничение на свойство *имеетНачинку*, которое называется аксиомой замыкания.

Пусть на свойство P наложены ограничения существования вида $P \text{ some } C1, P \text{ some } C2, \dots, P \text{ some } Cn$.

Закрывающая аксиома или аксиома замыкания на свойство (closure axiom) P это универсальное ограничение вида $P \text{ only } C$, где C – объединение классов $C1, C2, \dots, Cn$.

Для класса *МаргаритаПицца* аксиома замыкания будет записываться следующим образом: *имеетНачинку only (МоцареллаНачинка or ПомидорНачинка)*.

Упражнение 30.

Упражнение 31. Добавление аксиомы замыкания для класса *МаргаритаПицца*.

1. Выберите класс *МаргаритаПицца* в иерархии классов.
2. Нажмите кнопку  (Добавить) рядом с **SubClass Of** в окне области окна **Description:МаргаритаПицца**.
3. В появившемся окне выберите вкладку **Object restriction creator**.
4. В левой части окна (**Restricted property**) из иерархии свойств выберите свойство *имеетНачинку*.
5. В правой части окна (**Restriction filler**) из иерархии классов выделите сразу два класса: *МоцареллаНачинка* и *ПомидорНачинка*.
6. Убедитесь, что в нижней части окна в области **Restriction type** выбрано ограничение **Only** и нажмите **OK**.
7. Убедитесь, что описание класса *МаргаритаПицца* выглядит, как показано на рис. 5.16.

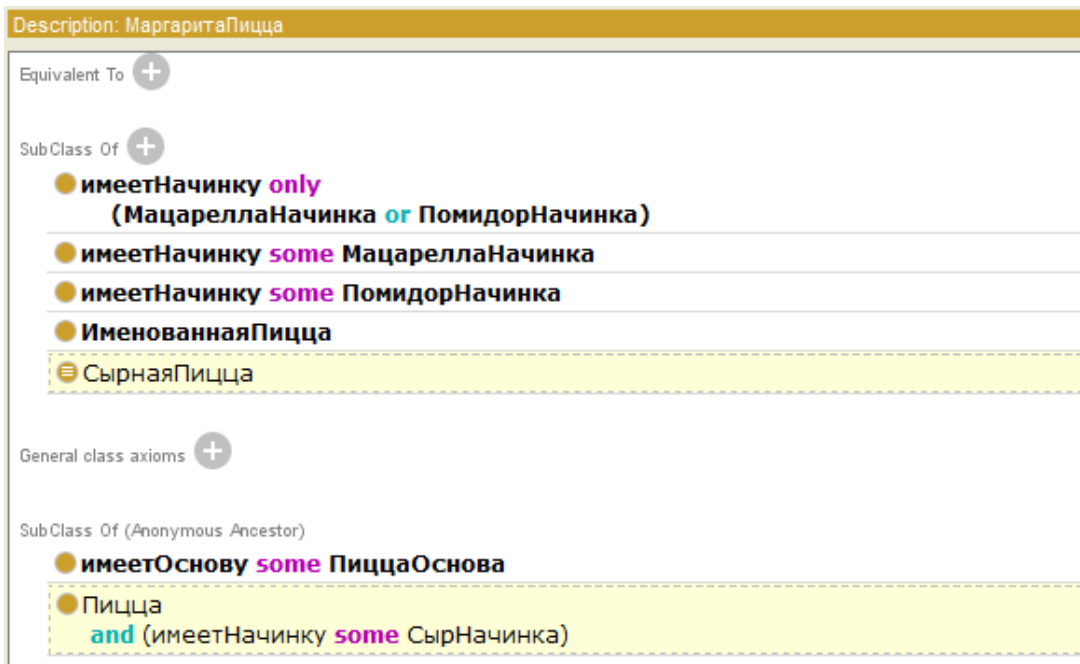


Рисунок 5.16. Окно Description:МаргаритаПицца

Такое описание класса *МаргаритаПицца* означает, что если индивид является членом класса *МаргаритаПицца*, то он должен иметь, по крайней мере, одну начинку вида *МоцареллаНачинка* и, по крайней мере, одну начинку вида *ПомидорНачинка* и не иметь больше никаких других начинок.

Распространённой ошибкой в данном случае является использование универсального ограничения вместо закрывающей аксиомы. Если мы опишем класс только с помощью универсального ограничения *имеетНачинку only (МоцареллаНачинка or ПомидорНачинка)*, но при этом не укажем ограничения существования *имеетНачинку some МоцареллаНачинка* и *имеетНачинку some ПомидорНачинка* это будет означать, что пицца, не имеющая никакой начинки, также является пиццей Маргарита.

Упражнение 32. Добавление аксиом замыкания для классов *СохоПицца*, *АмериканоПицца* и *АмериканоХотПицца*

1. Добавьте самостоятельно аксиому замыкания для классов *СохоПицца*, *АмериканоПицца* и *АмериканоХотПицца*.
2. Запустите машину вывода и сравните созданную и выведенную онтологию (рис 5.17).
3. Убедитесь, что классы *МаргаритаПицца* и *СохоПицца* классифицировались как подклассы класса *ВегетаринскаяПицца*.

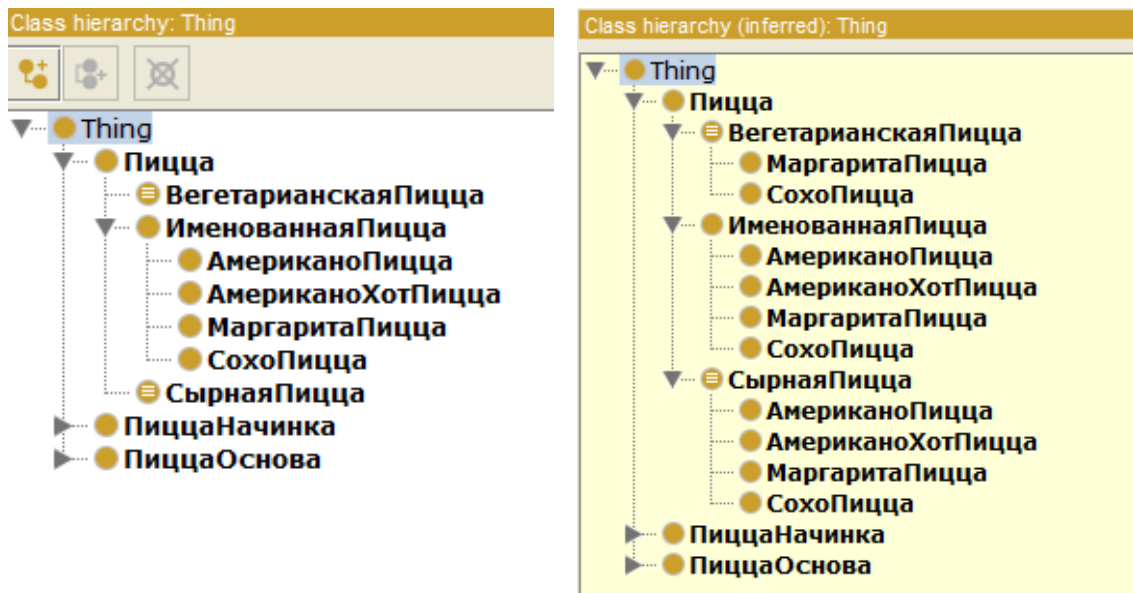


Рисунок 5.17. Сравнение созданной и выведенной онтологии

5.7. Паттерн проектирования «Разделение значений»

В этом разделе мы создадим так называемые разделения значений (value partition), которые используются для улучшения описания классов в онтологии. «Разделение значений» представляет собой шаблон (паттерн) проектирования и не является частью OWL или какого-либо другого языка онтологий. Паттерн проектирования онтологии являются аналогом паттерна проектирования в объектно-ориентированном программировании, т.е. представляет собой предлагаемое экспертами решение проблемы проектирования в рамках некоторого часто возникающего контекста. Шаблон проектирования «Разделение значений» служит для улучшения описания классов. В онтологии пицц создадим разделение значений с названием *ПряностьРазделениеЗначений*, для описания количества специй, которые используются в том или ином виде пиццы. Шаблон «Разделение значений» ограничивает диапазон возможных значений (т.е. исчерпывающий перечень значений), например, *ПряностьРазделениеЗначений* будет ограничивать диапазон значений: *Неострая*, *Средняя*, *Острая*.

Создание паттерна «Разделение значений» в языке OWL осуществляется с помощью следующих шагов.

1. Создание класса *РазделениеЗначений* для представления паттерна «Разделение значений». Например, создание класса *ПряностьРазделениеЗначений*.

2. Создание непересекающихся подклассов созданного класса для представления всевозможных диапазонов значений, например, создание

следующих непересекающихся подклассов класса
ПряностьРазделениеЗначений: Неострая, Средняя, Острая.

3. Создание функционального объектного свойства с диапазоном – классом *РазделениеЗначений*, например, для онтологии пицц можно создать свойства *имеетПряность*. с диапазоном *ПряностьРазделениеЗначений*.

4. Описание покрывающей аксиомы (covering axiom) для указания, что список значений является исчерпывающим.

Упражнение 33. Создание паттерна *РазделениеЗначений* для представления пряности пицц.

1. Создайте класс *РазделениеЗначений* как подкласс класса *Thing*.
2. Создайте класс *ПряностьРазделениеЗначений* как подкласс класса *РазделениеЗначений*.
3. Для класса *ПряностьРазделениеЗначений* создайте непересекающиеся подклассы: *Неострая, Средняя, Острая*.
4. Создайте объектное свойство *имеетПряность* как подсвойства свойства *topObjectProperty*.
5. Убедитесь, что иерархия классов выглядит, как показано на рис. 5.18.
6. Для свойства *имеетПряность* укажите диапазон *ПряностьРазделениеЗначений*.
7. Для свойства *имеетПряность* укажите, что оно является функциональным.
8. Убедитесь, что описание свойства выглядит, как показано на рис. 5.19.

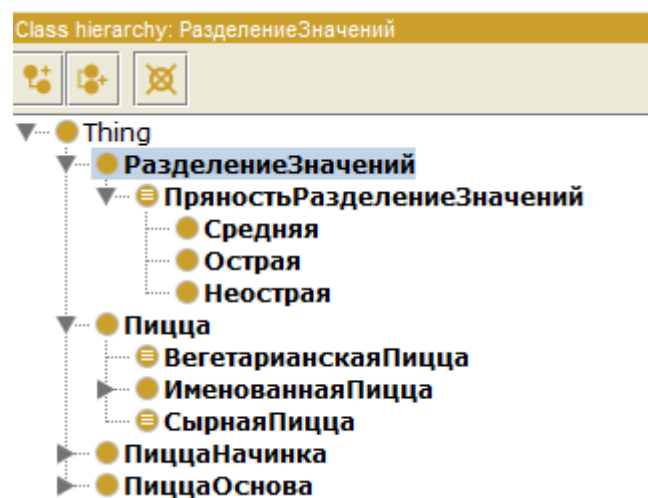


Рисунок 5.18. Иерархия классов

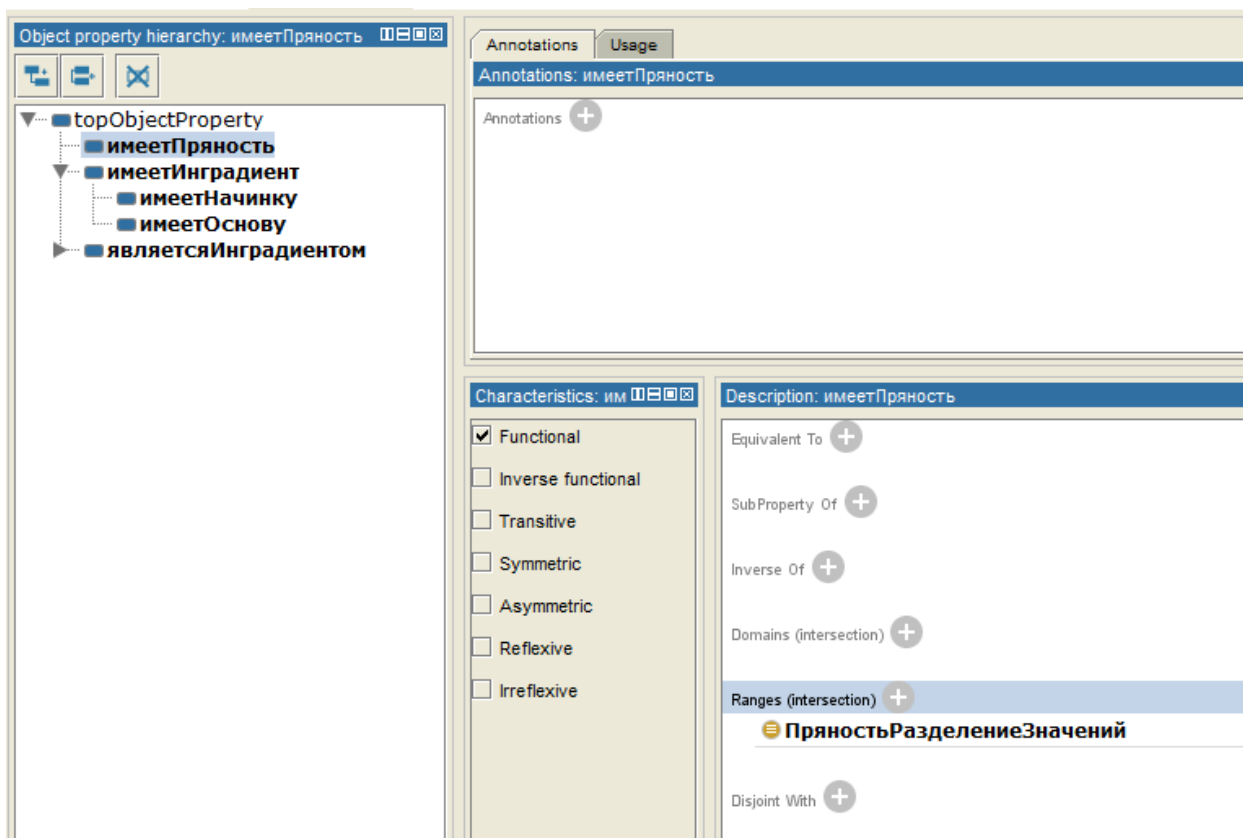


Рисунок 5.19. Описание свойства имеетПряность

Последним шагом создания паттерна *РазделениеЗначения* является создание покрывающей аксиомы. Покрывающая аксиома состоит из двух частей: класса, который «покрывается» и классов, которые образуют покрытие. Например, предположим, что имеются три класса A , B , C , причём классы B и C являются подклассами класса A . Пусть имеется покрывающая аксиома для класса A с покрывающими классами B и C . Это означает, что любой член класса A должен быть членом классов B и/или C . Если классы B и C не пересекаются, это означает, что любой член класса A должен быть членом либо класса B , либо класса C . Напомним, что в общем случае, несмотря на то, что классы B и C являются подклассами класса A , член класса A может не являться членами класса B или C .

В Protégé чтобы создать покрывающую аксиому для покрываемого класса, надо объявить эквивалентный ему класс как объединение покрывающих классов, т.е. для класса A надо объявить эквивалентный класс как объединение классов B и C .

В нашем примере для класса *ПряностьРазделениеЗначений* надо объявить эквивалентный класс как объединение классов *Неострая*, *Средняя*, *Острая*.

Упражнение 34. Добавление покрывающей аксиомы для класса *ПряностьРазделениеЗначений*

1. Выберите класс *ПряностьРазделениеЗначений* в иерархии классов.
2. Нажмите кнопку  (Добавить) рядом с **Equivalent of** в окне области окна **Description: ПряностьРазделениеЗначений**.
3. Появившемся окне выберите вкладку **Class expression editor**, напечатайте ограничение *Острая or Средняя or Неострая* и нажмите **ОК**.
4. Убедитесь, что описание класса выглядит, как показано на рис. 5.20.

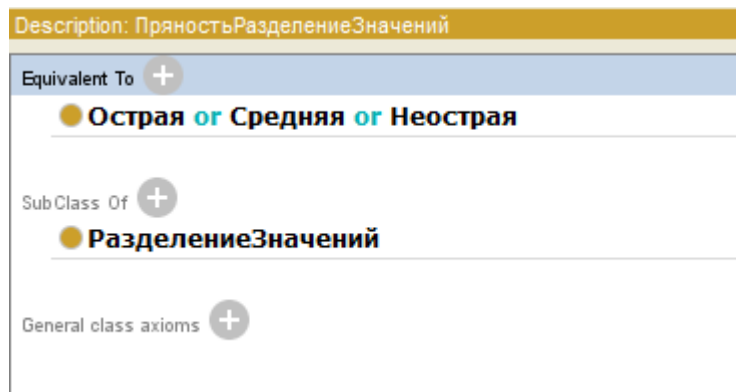


Рисунок 5.20. Окно Description: ПряностьРазделениеЗначений

Класс *ПряностьРазделениеЗначений* можно использовать для описания начинок пиццы. Для этого надо добавить ограничение существования к каждому виду начинки пиццы, которое устанавливает ее пряность, вида *имеетПряность some ПряностьРазделениеЗначений*, где *ПряностьРазделениеЗначений* будет иметь одно из значений: *Неострая*, *Средняя*, *Острая*. Чтобы ускорить данный процесс, рекомендуется использовать плагин Matrix. Это плагин позволит представлять онтологию в виде различных таблиц и упрощает создание некоторых ограничений.

Упражнение 35. Инсталляция плагинов в Protégé

1. Выберите пункт **Preferences (Параметры)** из меню **File**.
2. В появившемся окне выберите вкладку **Plugins** (рис 5.21).
3. Для того чтобы обновить существующие плагины, нажмите кнопку **Check for updates now (Нажмите для обновления сейчас)** в секции **Auto update (Автоматическое обновление)**.
4. Для того чтобы установить новые плагины, нажмите кнопку **Check for updates now** в секции **Plugin registry (Регистрация плагинов)**.

5. В появившемся окне выберите нужные плагины, в частности **Matrix Plugin** (рис. 5.22).
6. Убедитесь, что выбрана опция **Always check for updates on startup** (Всегда проверять обновления при запуске), и нажмите кнопку **Install** (Установить). Обратите внимание, что изменения вступят в силу только после перезапуска Protégé.

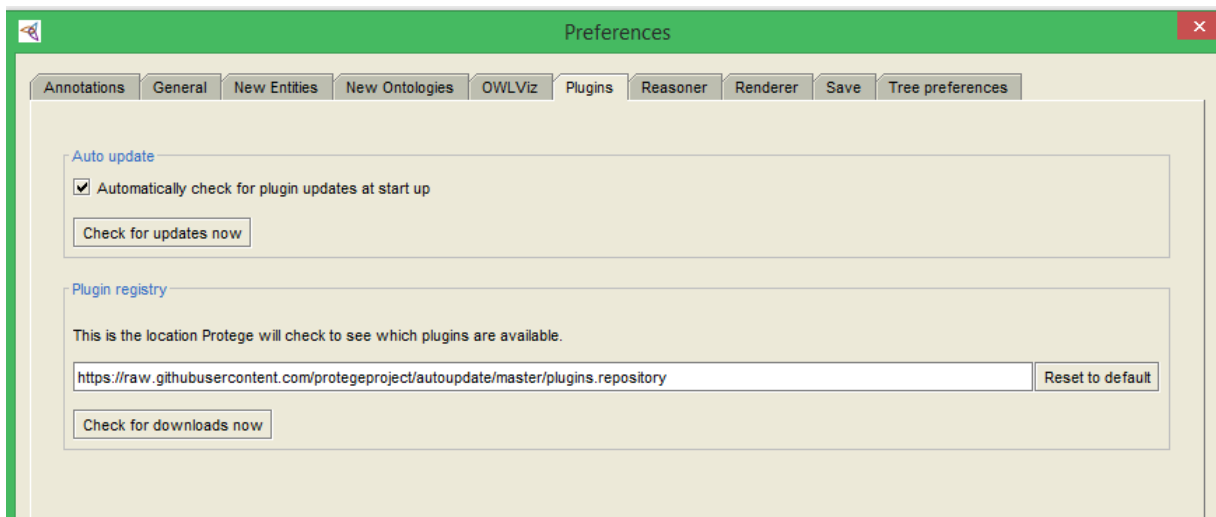


Рисунок 5.21. Вкладка Plugins окна Preference

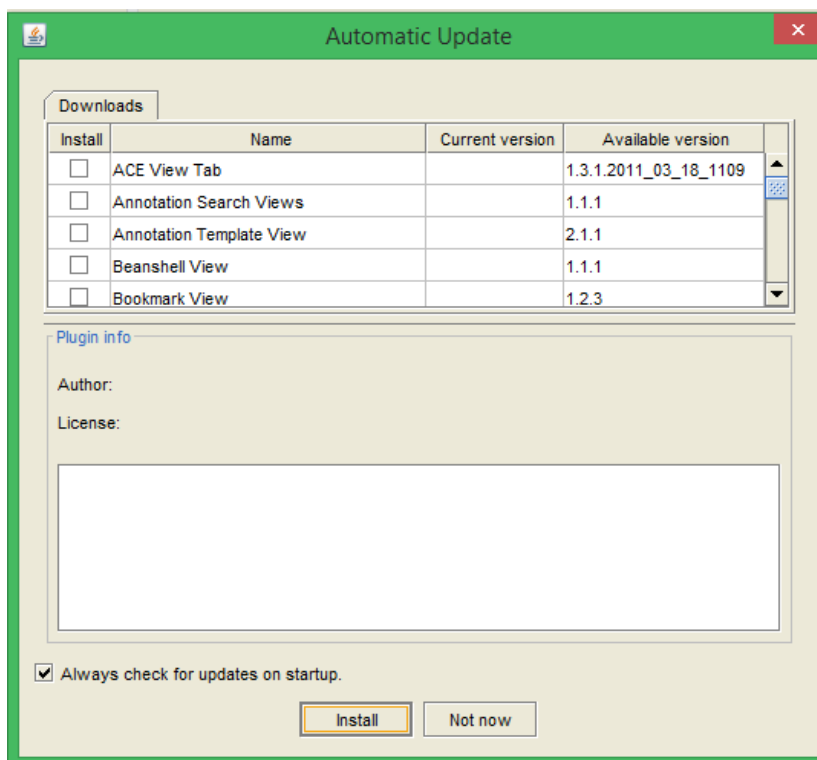


Рисунок 5.22. Окно Automatic Update

Упражнение 36. Использование плагина Matrix для добавления пряности к описанию начинок пицц

1. В меню **Windows** выберите **Tabs** ⇒ **Class Matrix**. В главном окне Protégé должна появиться вкладка **Class matrix**.
2. Перейдите на вкладку **Class matrix** (рис 5.23).
3. В левой части окна выберите в иерархии свойство *имеетПряность* и перетащите его в пустую область окна **Class Matrix**.
4. Для каждого класса подкласса *ПиццаНачинка*, который не имеет подклассов, введите одно из значений свойства *имеетПряность*: *Неострая*, *Средняя*, *Острая* (рис. 5.24). Для ускорения ввода можно использовать подстановку с помощью сочетаний клавиш **Ctrl+Enter**.
5. Для одного из классов на вкладке **Classes** убедитесь, что в его описании появилось соответствующее ограничение существования. Например, для класса *ХалапеньоПерецНачинка* ограничение существования будет выглядеть следующим образом: *имеетПряность some Острая* (рис. 5.25)

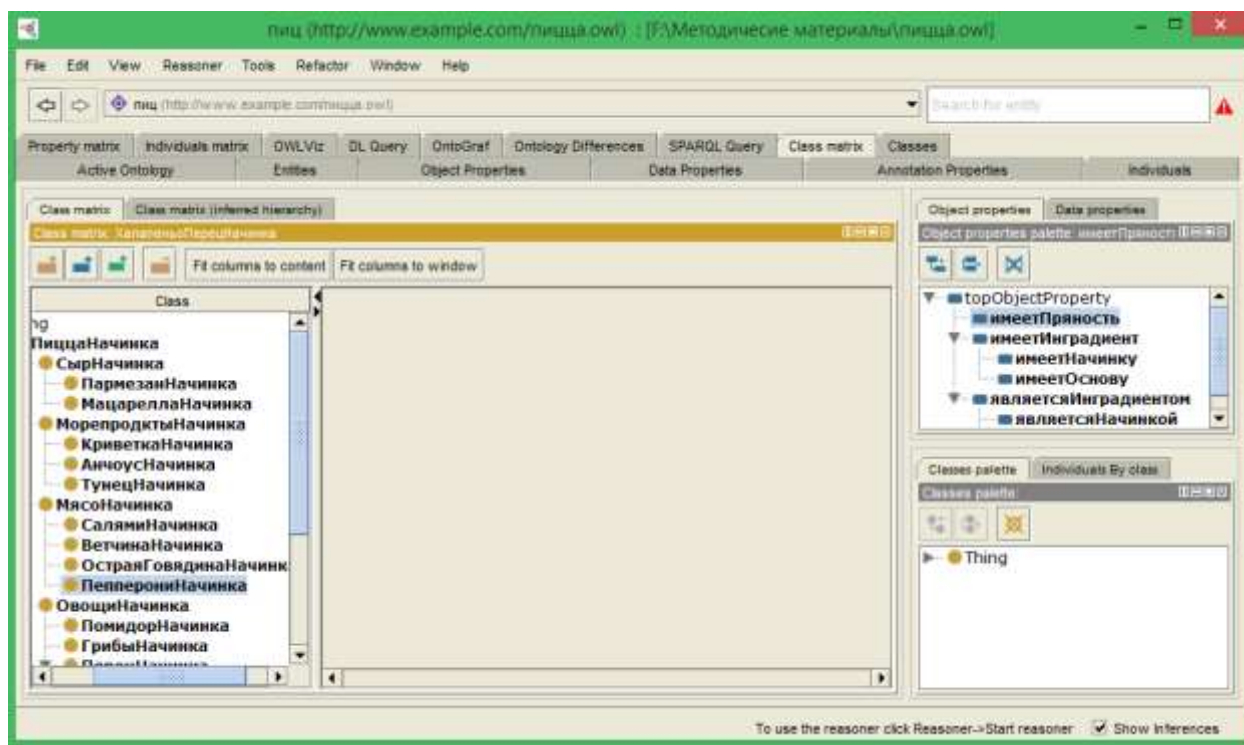


Рисунок 5.23. Вкладка Class matrix

Class matrix	
Class matrix (inferred hierarchy)	
Class matrix: ХалапеньоПерецНачинка	
Fit columns to content Fit columns to window	
Class	имеетПряность (some)
Thing	
ПиццаНачинка	
СырНачинка	
ПармезанНачинка	Средняя
МацареллаНачинка	Средняя
МорепродуктыНачинка	
КриветкаНачинка	Неострая
АнчоусНачинка	Средняя
ТунецНачинка	Неострая
МясоНачинка	
СалямиНачинка	Средняя
ВетчинаНачинка	Средняя
ОстраяГовядинаНачинка	Острая
ПепперониНачинка	Острая
ОвощиНачинка	
ПомидорНачинка	Неострая
ГрибыНачинка	Неострая
ПерецНачинка	
ХалапеньоПерецНачинка	Острая
КрасыйПерецНачинка	Средняя
ЗеленыйПерецНачинка	Неострая
ОливкиНачинка	Неострая
ЛукНачинка	Средняя
КаперсыНачинка	Острая

Рисунок 5.24. Вкладка Class matrix: значения свойства имеетПряность

Description: ХалапеньоПерецНачинка	
Equivalent To	+
SubClass Of	+
	имеетПряность some Острая ПерецНачинка
General class axioms	+

Рисунок 5.25. Окно Description: ХалапеньоПерецНачинка

Чтобы продемонстрировать использование паттерна «Разделение значений», создадим класс *ОстраяПицца*, который описывает пиццы, которые имеют хотя бы одну острую начинку. Для этого можно создать класс *ОстраяПиццаНачинка* (как подкласс класса *ПиццаНачинка*) с ограничением *имеетПряность some Острая*. И затем для класса *ОстраяПицца* указать ограничение *имеетНачинку some ОстраяПиццаНачинка*.

Однако то же самое можно сделать и без создания именованного класса *ОстраяПиццаНачинка*. Для этого нужно для класса *ОстраяПицца*

создать *имеетНачинку some (ПиццаНачинка and (имеетПряность some Острая))*). Именно это ограничение создает анонимный класс, эквивалентный именованному классу *ОстраяПиццаНачинка*.

Упражнение 37. Создание класса ОстраяПицца

1. Создайте класс *ОстраяПицца* как подкласс *Пицца*.
2. Для класса *ОстраяПицца* добавьте ограничение на свойство: *имеетНачинку some (ПиццаНачинка and (имеетПряность some Острая))*.
3. Преобразуйте класс *ОстраяПицца* в определённый.
4. Убедитесь, что описание класса выглядит, как показано на рис. 5.26.
5. Запустите машину логического вывода.
6. Убедитесь, что классы *АмериканоПицца* и *АмериканоХотПицца* проклассифицировались как подклассы класса *ОстраяПицца*, так как содержат, по крайней мере, одну острую начинку.
- 7.

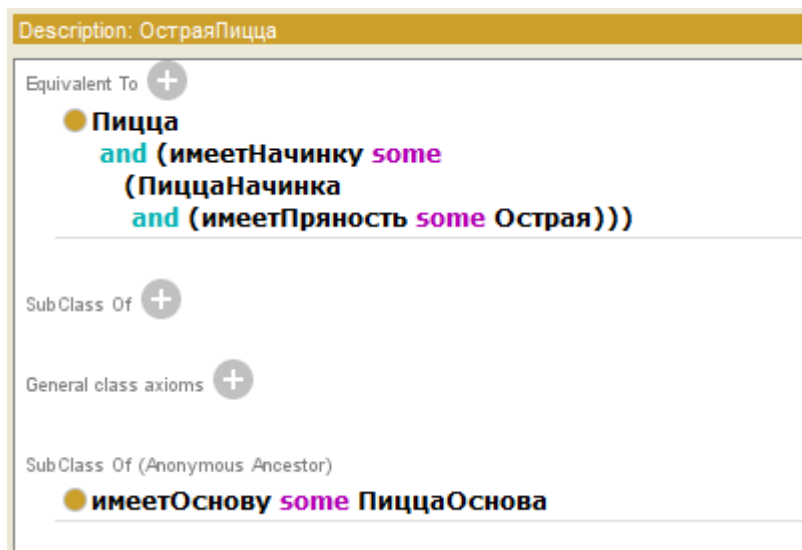


Рисунок 5.26. Окно Description:ОстраяПицца

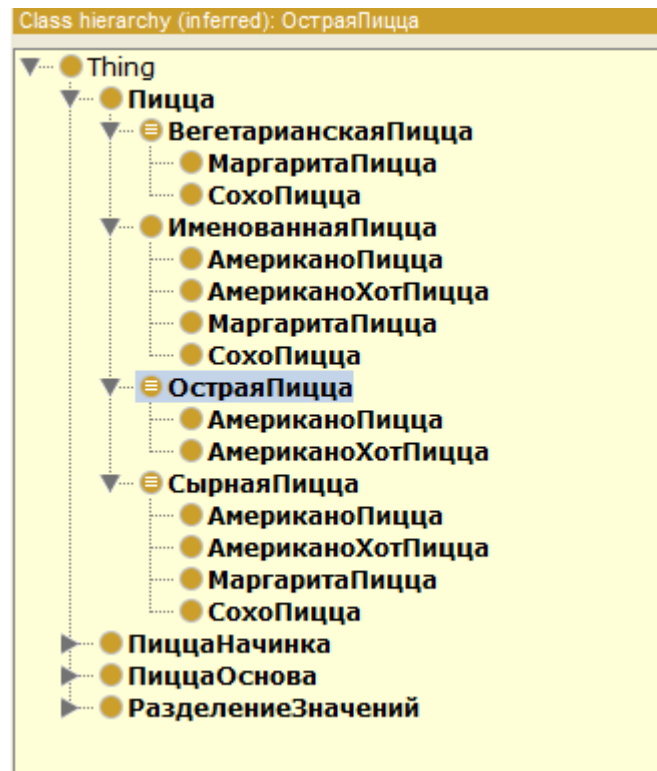


Рисунок 5.27. Выведенная иерархия классов

5.8. Ограничения кардинальности

В OWL можно описать классы индивидов, для которых ограничено число связей, в которых они могут участвовать. Например, можно указать, что каждый индивид класса *Дисциплина* участвует как минимум в одной связи *читаетсяПреподавателем*, т.е. каждая дисциплина читается как минимум одним преподавателем. Такие ограничения называются ограничениями кардинальности.

Ограничение кардинальности на свойство P описывает класс A для которого справедливо следующее. Каждый индивид класса A может участвовать в ограниченном числе связей P .

Уточнённое ограничение кардинальности на свойство P описывает класс A , для которого справедливо следующее. Каждый индивид класса A может участвовать в ограниченном числе связей P с индивидами заданного класса или литеральными значениями.

Ограничения кардинальности, в свою очередь, делятся на три подвида.

Максимальное ограничение кардинальности (**at most**) указывает максимальное число связей P , в котором могут участвовать индивиды описываемого класса.

Максимальное ограничение кардинальности (**at least**) указывает максимальное число связей, в котором могут участвовать индивиды описываемого класса.

Уточнённое ограничение кардинальности (**exactly**) указывает точное число связей, в котором могут участвовать индивиды описываемого класса.

Пример максимального ограничения кардинальности: каждый индивид класса *УчебнаяГруппа* участвует не более чем в 25 связях *включаетСтудента*, т.е. в каждой группе могут учиться не более 25 студентов. В Protégé данное ограничение будет записано в виде: *включаетСтудента max 25 Студент*.

Пример минимального ограничения кардинальности: каждый индивид класса *МногодетныйРодитель* участвует как минимум в трёх связях *имеетРебенка*, т.е. каждый многодетный родитель имеет не менее трёх детей. В Protégé данное ограничение будет записано в виде: *имеетРебенка min 3 Человек*.

Пример точного ограничения кардинальности: каждый индивид класса *Человек* участвует ровно в двух связях *имеетРодителя*, т.е. каждый имеет двух родителей. В Protégé данное ограничение будет записано в виде: *имеетРодителя exactly 2 Человек*.

Заметим, что ограничение кардинальности учитывает количество *различных* связей, в которых участвует индивид. Две связи *индивидаA свойствоP индивидB* и *индивидаA свойствоP индивидC* считаются различными, если *индивидB* и *индивидC* – это различные индивиды.

Добавим ограничения кардинальности в онтологию пицц: создадим класс *ИнтереснаяПицца*, описывающий пиццы, имеющие, по крайней мере, три разные начинки.

Упражнение 38. Создание класса *ИнтереснаяПицца*

1. Создайте класс *ИнтереснаяПицца* как подкласс класса *Пицца*.
2. Нажмите кнопку  (Добавить) рядом с **SubClass Of** в окне области окна **Description:ИнтереснаяПицца**.
3. В появившемся окне выберите вкладку **Class expression editor**.
4. Введите ограничение *имеетНачинку exactly 3* и нажмите **OK**.

5. Преобразуйте класс *ИнтереснаяПицца* в определённый.
6. Убедитесь, что описание класса *ИнтереснаяПицца*, выглядит, как показано на рис. 5.28.
7. Запустите машину логического вывода.
8. Убедитесь, что классы *АмериканоПицца*, *АмериканоХотПицца* и *СохоПицца* проклассифицировались как подклассы класса *ИнтереснаяПицца*, так как содержат, по крайней мере, три вида начинок (рис. 5.29).

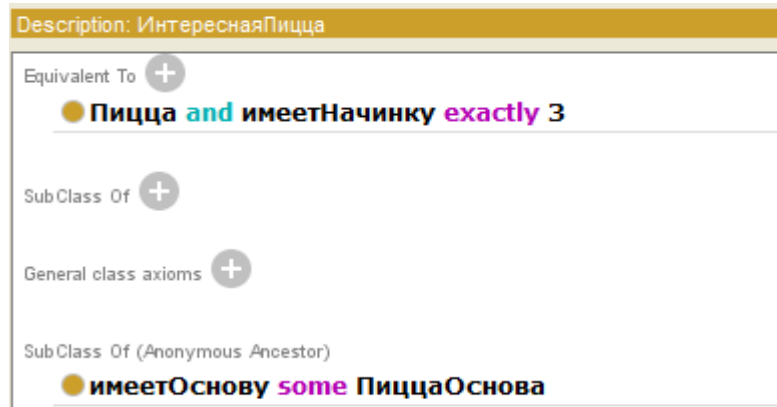


Рисунок 5.28. Окно Description:ИнтереснаяПицца

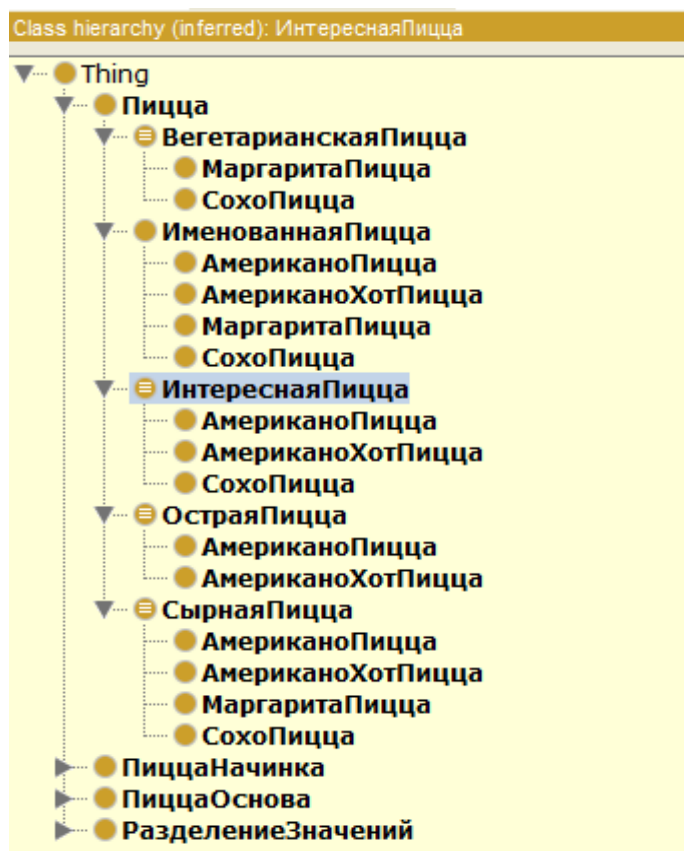


Рисунок 5.29. Выведенная иерархия классов

Описанное ограничение кардинальности является неуточненным, т.е. мы определили, что каждый индивид класса имеет три начинки, но не уточнили, какие именно. Создадим уточнённое ограничение кардинальности, т.е. ограничение, в котором указываются классы, на которые оно распространяется, а именно опишем класс *ЧетыреСыраПицца*, индивиды которого имеют точно 4 вида сырной начинки.

Упражнение 39. Создание класса *ЧетыреСыраПицца*

1. Создайте класс *ЧетыреСыраПицца* как подкласс класса *ИменованнаяПицца*.
2. Нажмите кнопку  (Добавить) рядом с **SubClass Of** в окне области окна **Description: ЧетыреСыраПицца**.
3. В появившемся окне выберите вкладку **Object restriction creator**.
4. В левой части окна (**Restricted property**) из иерархии свойств выберите свойство **имеетНачинку**.
5. В правой части окна (**Restriction filler**) из иерархии классов выделите класс *СырНачинка*.
6. Выберите в нижней части окна в области **Restriction type** ограничение **Exactly**, в области **Cardinality** введите число 4 и нажмите ОК.
7. Убедитесь, что описание класса *ЧетыреСыраПицца* выглядит, как показано на рис. 5.30.

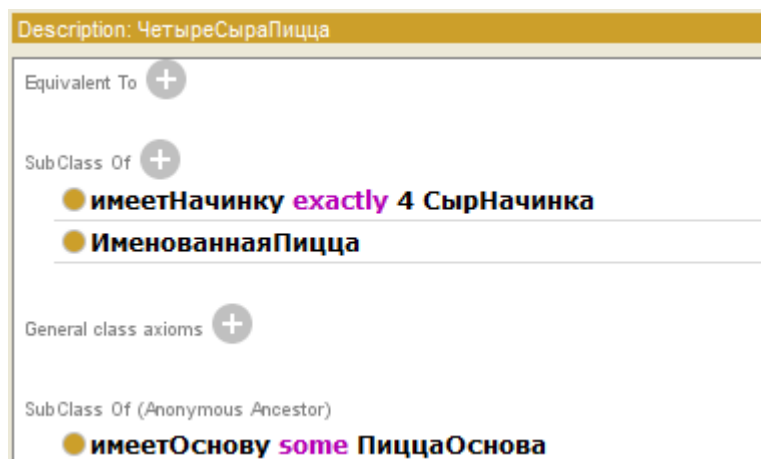


Рисунок 5.30. Окно Description:ЧетыреСыраПицца

Данное описание класса *ЧетыреСыраПицца* говорит о том, что данный вид пиццы имеет ровно четыре сырные начинки, но это не значит,

что данная пицца не имеет других начинок. Если мы хотим сказать, что данная пицца содержит только 4 вида сырных начинок и не содержит никаких других начинок, то к ограничению кардинальности надо добавить универсальное ограничение *имеетНачинку Only СырНачинка*.

Любое неуточненное ограничение кардинальности эквивалентно уточнённому ограничению с уточняющим классом Thing. Например, ограничение *имеетНачинку min 3* равносильно ограничению *имеетНачинку min 3 Thing*.

Глава 6. СВОЙСТВА ТИПА ДАННЫХ И СОЗДАНИЕ ИНДИВИДОВ

6.1. Свойства типа данных

Свойства типа данных связывают индивиды классов с некоторыми литеральными значениями. Примером свойств типа данных могут быть свойства *имеетФамилию*, *имеетРост*, примером утверждений об индивидах – утверждения *Человек1 имеетФамилию Иванов*, *Человек1 имеетРост 176*. В первом случае в качестве литерального значения выступает строка, во втором – целое число. Единственно возможная характеристика для свойств типа данных – функциональность. Два вышеприведенных свойства являются функциональными, так как у человека может быть только одна фамилия и только одно значение роста.

В редакторе Protégé работа со свойствами типа данных происходит на отдельной вкладке программы **Data Property**. Свойства типа данных, так же как и объектные свойства, организуются в иерархию, в данном случае с единым корневым свойством *topDataProperty*.

Упражнение 40. Создание свойства типа данных *имеетКалорийность*

1. Перейдите на вкладку **Data Property**.
2. Создайте свойство *имеетКалорийность* как подсвойство свойства *topDataProperty*.
3. Укажите, что свойство является функциональным.

6.2. Создание индивидов

После того как созданы классы, свойства и аксиомы, можно приступить к наполнению онтологии индивидами. Можно очень приблизительно сравнить индивид онтологии с экземпляром таблицы базы данных. Наполнение онтологии, как и наполнение базы данных, – это трудоемкий процесс, требующий времени и внимательности оператора.

Предположим, что необходимо хранить данные о конкретном человеке. При заполнении традиционной базы данных для этого требуется заполнить одну строку таблицы *Человек* (или несколько строк в разных таблицах, если, например, требуется указать, что человек является владельцем автомобиля). В онтологии для этого необходимо создать конкретный индивид класса *Человек* и вручную сделать о нем несколько

утверждений, т.е. связать его существующими свойствами онтологии с индивидами других классов или литеральными значениями. Таким образом, процесс наполнения онтологии индивидами еще более трудоемок, чем процесс заполнения базы данных. Поэтому на практике для заполнения баз данных и наполнения онтологий первоначальным массивом данных используют специальные программы – конверторы из различных источников данных (других баз данных, электронных таблиц и т.д.). В данном учебном пособии рассматривается только ручной способ добавления индивидов.

Работа с индивидами в Protégé происходит на отдельной вкладке программы **Individuals (Индивиды)**.

Упражнение 41. Создание индивидов класса Пицца

1. Перейдите на вкладку **Individuals (Индивиды)** (рис 6.1).
2. В окне **Class hierarchy** в иерархии классов выберите класс *МаргаритаПицца*.
3. В окне **Members list** (Список членов) окна нажмите на кнопку  (Добавить индивид), введите имя *ПримерМаргаритаПицца* и нажмите **OK**.
4. В окне **Property assertion (Утверждение о свойствах)** нажмите на кнопку  рядом с **Data property assertion (Утверждение о свойствах типа данных)**.
5. В появившемся окне (рис. 6.2) в левой части выберите свойство *имеетКалорийность*, а в правой введите значение этого свойства 263 и нажмите **OK**.
6. Создайте ещё несколько экземпляров пицц с различной калорийностью: *ПримерАмериканоПицца* с калорийностью 380, *ПримерАмериканоХотПицца* с калорийностью 550, *ПримерЧетыреСыраПицца* с калорийностью 723.

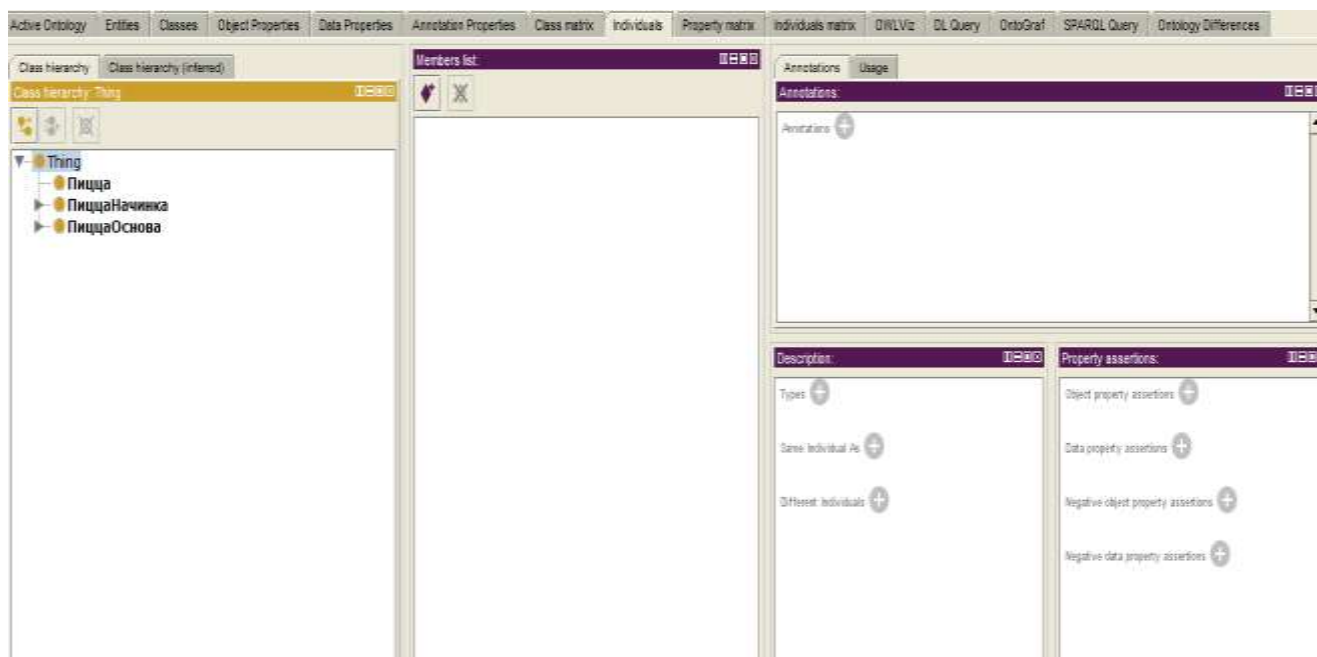


Рисунок 6.1. Вкладка Individuals

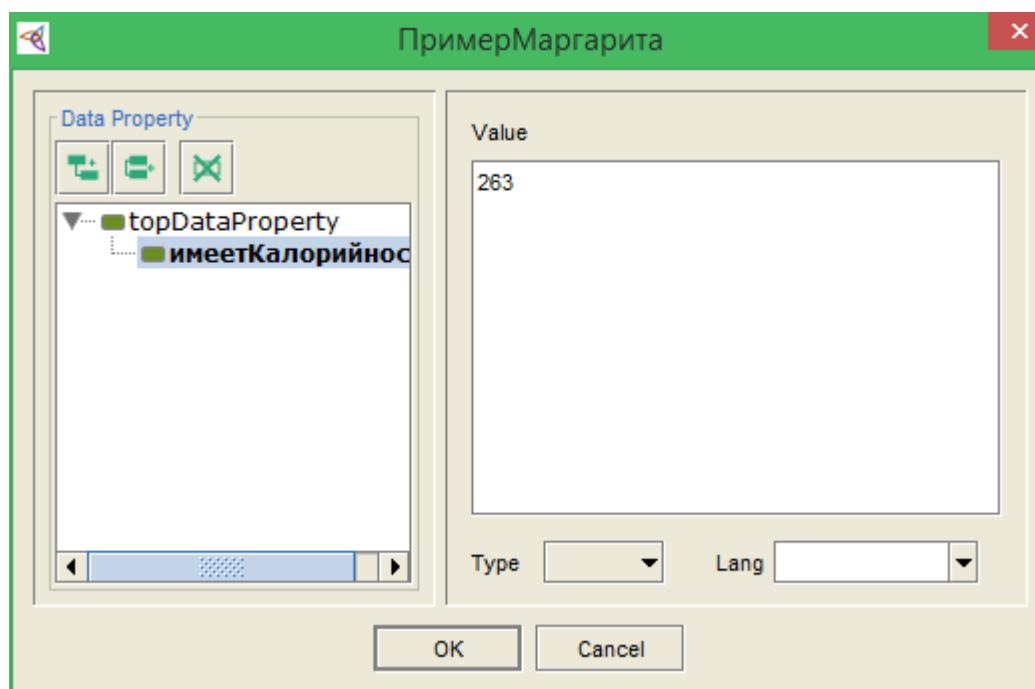


Рисунок 6.2. Значение свойства имеетКалорийность

Обратите внимание, что на вкладке **Individuals** в части окна **Description** (рис. 6.3) для выделенного индивида отображается класс, которому он принадлежит (в области **Type**). Заметим, что в общем случае индивид может принадлежать нескольким классам, при условии, что они не объявлены как непересекающиеся, например, некоторый человек может быть одновременно индивидом класса *Сотрудник* и класса *Женщина*.

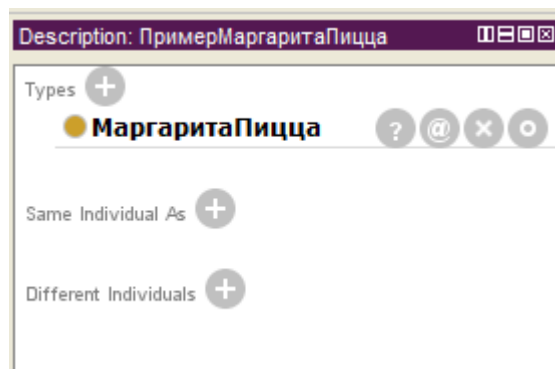


Рисунок 6.3. Окно Description: ПримерМаргарита

Секция **Same Individuals as** (Такой же индивид как) в окне **Description** крайне важна для использования онтологии в семантическом вебе. Именно она позволяет установить соответствие между индивидами из разных онтологий и обеспечивает большинство преимуществ онтологического моделирования знаний в вебе.

Секция **Different Individuals** (Различен с индивидами) позволяет сделать явное утверждение о различии индивидов. Это крайне важно для любой онтологии. Если явным образом не указывать различие индивидов, то машина логического вывода не сможет обнаружить многие противоречия в онтологии. Например, пусть в онтологии свойство *имеетМать* описано как функциональное (так как у любого человека может быть только одна мать) и пусть имеются два утверждения об индивиде Иван: *Иван имеетМать Ольгу* и *Иван имеетМать Олю*. Если явно не указать, что *Ольга* и *Оля* – это два разных индивида, то машина вывода не найдет этого логического противоречия. Еще один пример обнаружения противоречивости на основе указания различия индивидов приведен в п. 6.5.

Как уже упоминалось, после наполнения онтологии индивидами ее надо проверить на согласованность с помощью машины логического вывода.

Упражнение 42. Проверка онтологии на согласованность

1. Проверьте онтологию на согласованность.
2. Добавьте ещё одно значение свойства *имеетКалорийность* для индивида *ПримерМаргаритаПицца*, например 300.
3. Опять проверьте онтологию на согласованность. Убедитесь, что индивид *ПримерМаргаритаПицца* выделен красным, то есть рассматривается как противоречивый. Проанализируйте, почему?
4. Верните онтологию в согласованное состояние.

6.3. Ограничения на диапазон значений свойств типа данных

В языке OWL для свойств типа данных существует специальный вид ограничения на диапазон значений. Добавим подобное ограничение. Каждая пицца имеет, по крайней мере, одно значение калорийности и это значение – целое число. Учитывая, что свойство *имеетКалорийность* объявлено нами как функциональное, каждая пицца будет иметь точно одно значение этого свойства.

Упражнение 43. Создание ограничения на свойство *имеетКалорийность*

1. Выберите вкладку **Classes**.
2. В иерархии классов выберите класс *Пицца*.
3. В окне **Description:Пицца** нажмите кнопку  рядом с **SubClass Of**.
4. В появившемся окне выберите вкладку **Data restriction creator**.
5. В левой части окна выберите свойство *имеетКалорийность*, в правой – тип данных *integer*, убедитесь, что в области **Restriction type** указано слово **some** (рис 6.4) и нажмите **OK**.

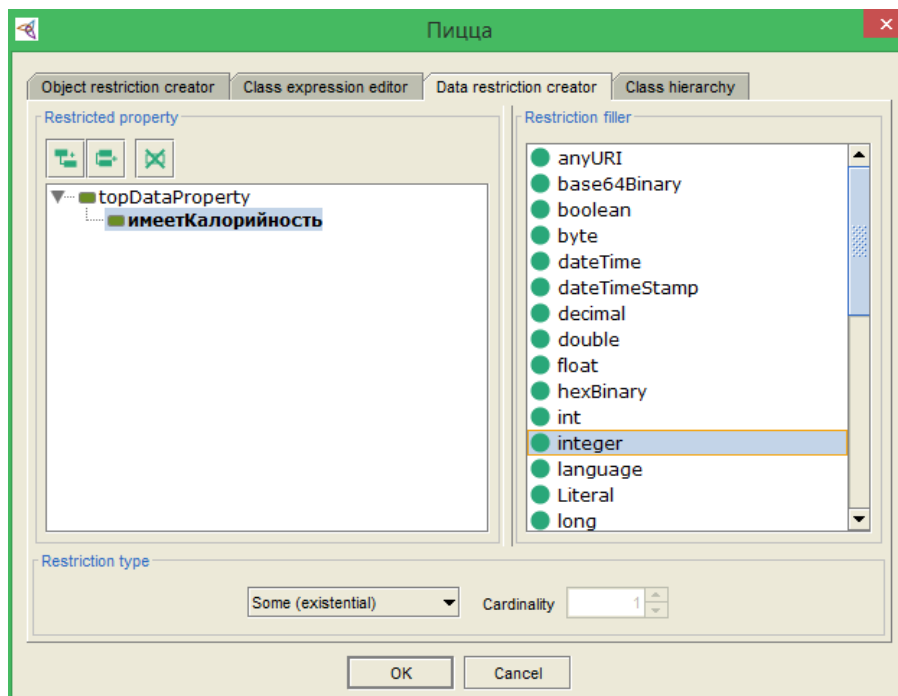


Рисунок 6.4. Создание ограничения на свойство

При создании подобного рода ограничений можно указать не только тип данных, которому должны принадлежать значения свойства, но и непосредственно диапазон значений. Покажем, как благодаря указанию

ограничения на диапазон значений свойства *имеетКалорийность* можно разделить все пиццы на высококалорийные и низкокалорийные.

Упражнение 44. Создание классов *ВысококалорийнаяПицца* и *НизкокалорийнаяПицца*

1. Создайте класс *ВысококалорийнаяПицца* как подкласс класса *Пицца*.
2. Нажмите кнопку  рядом с **SubClass Of** области окна **Description:ВысококалорийнаяПицца**.
3. В появившемся окне выберите вкладку **Class expression editor**, введите ограничение *имеетКалорийность some integer [≥ 400]* и нажмите **OK**.
4. Преобразуйте класс *ВысококалорийнаяПицца* в определённый.
5. Убедитесь, что описание класса *ВысококалорийнаяПицца* выглядит, как на рис. 6.4.
6. Аналогичным образом создайте класс *НизкокалорийнаяПицца*.

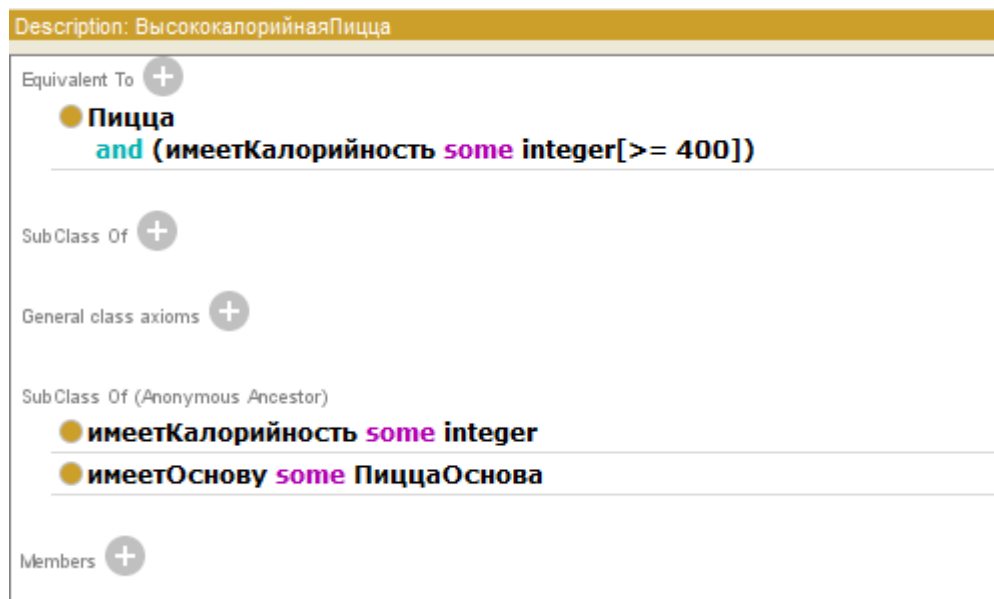


Рисунок 6.5. Окно Description:ВысококалорийнаяПицца

Упражнение 45. Автоматическая классификация сущностей

1. Запустите машину логического вывода.
2. Убедитесь, что все индивиды онтологии были классифицированы либо как экземпляры класса *ВысококалорийнаяПицца*, либо как экземпляры класса *НизкокалорийнаяПицца*. Например, описание класса *ВысококалорийнаяПицца* должно выглядеть, как на рис. 6.5.

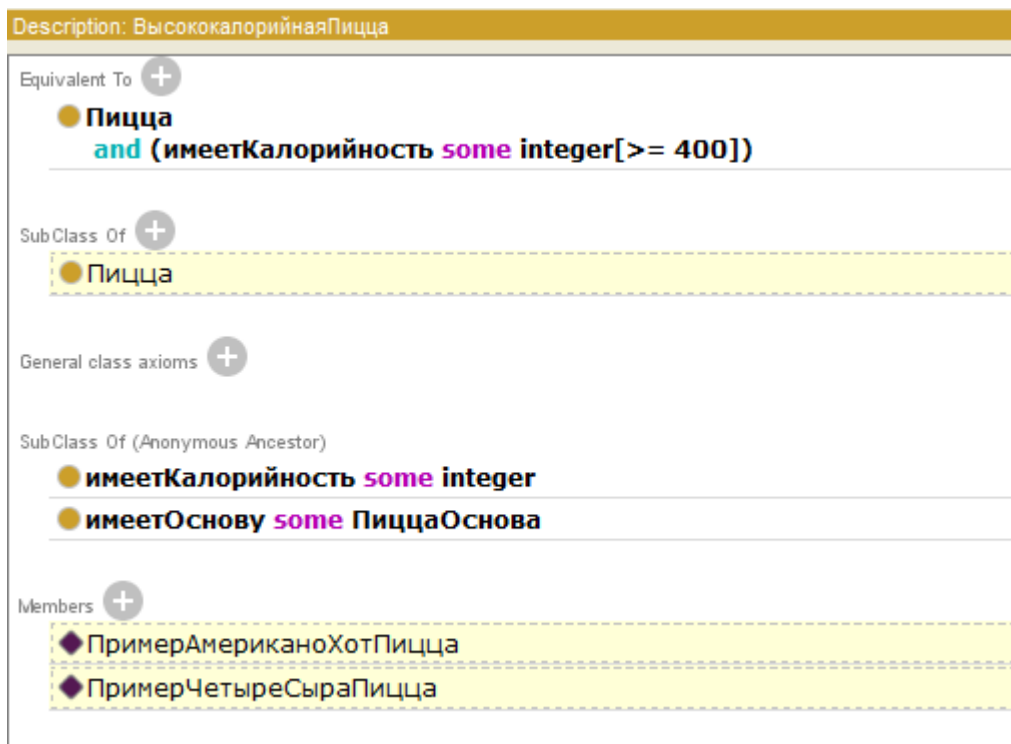


Рисунок 6.6. Окно Description:ВысококалорийнаяПицца

6.4. Ограничения на значения объектных свойств

И для объектных свойств, и для свойств типа данных можно создать ограничение, указав точное значение свойства. Для этого в редакторе Protégé используется слово **value** в качестве типа ограничения.

Продemonстрируем, как создать такого рода ограничение: в онтологии пицц укажем, что индивиды класса *МоцареллаНачинка* производятся только в Италии. Для этого необходимо создать класс *Страна* и объектное свойство *имеетСтрануПроисхождения*.

Упражнение 46. Создание ограничения на значения свойства *имеетСтрануПроисхождения*.

6. Создайте класс *Страна* как подкласс класса *Thing*.
7. Создайте индивиды класса *Страна*: *Америка*, *Великобритания*, *Италия*, *Франция*.
8. Создайте объектное свойство *имеетСтрануПроисхождения*.
9. Переключитесь на вкладку **Classes** и выберите класс *МоцареллаНачинка*.
10. Нажмите кнопку  рядом с **SubClass Of** области окна **Description:МоцарелаНачинка**.

11. В редакторе введите ограничение на свойство вида *имеетСтрануПроисхождения value Италия* и нажмите **ОК**.
12. Убедитесь, что описание класса *МоцареллаНачинка* выглядит так, как показано на рис. 6.7. Данное описание означает, что начинка из моцареллы – это сырная начинка, которая имеет среднюю остроту и производится в Италии.

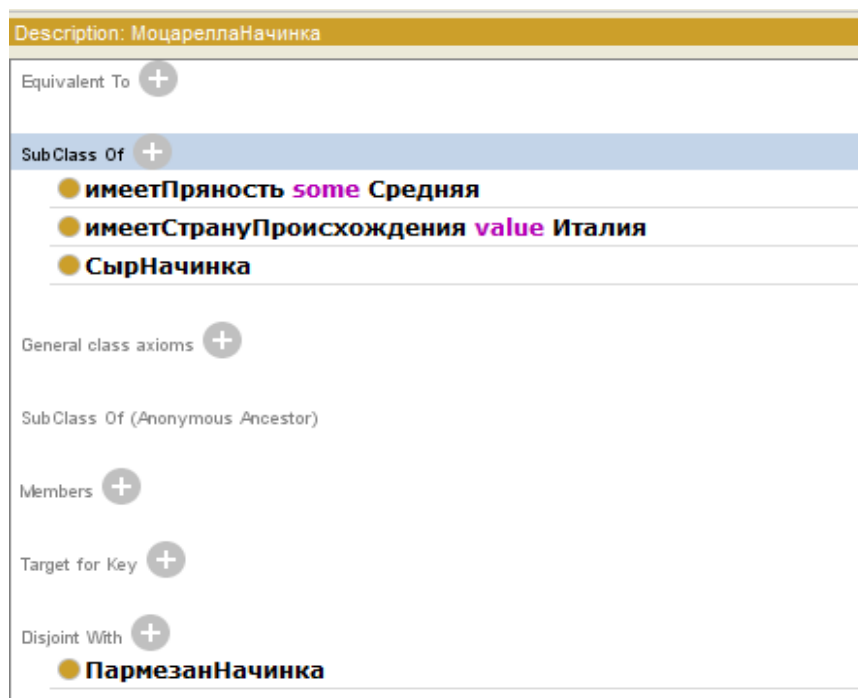


Рисунок 6.7. Окно Description:МоцареллаНачинка

6.5. Задание класса перечислением индивидов

Иногда возможно задать класс явным перечислением (списком) его индивидов. В этом случае невозможно будет создать индивид этого класса, не совпадающий с одним из индивидов из данного списка. Например, класс *ДеньНедели* может быть задан перечислением его индивидов {Понедельник, Вторник, Среда, Четверг, Пятница, Суббота, Воскресенье}.

Чтобы в редакторе Protégé описать класс явным перечислением индивидов, надо создать эквивалентный ему анонимный класс с заданным в редакторе ограничений списком индивидов.

Упражнение 47. Описание класса Страна перечислением его индивидов.

1. На вкладке **Classes** выделите класс *Страна*.

2. Нажмите кнопку  рядом с **Equivalent To** области окна **Description:Страна**.
3. В редакторе введите ограничение вида {Америка, Великобритания, Франция, Италия}. (Не забывайте, что для ускорения набора текста можно пользоваться горячими клавишами **Ctrl+Enter**). Нажмите **OK**.
4. Убедитесь, что описание класса *Страна* выглядит так, как показано на рис. 6.8. Это означает, что в разрабатываемой онтологии в качестве страны происхождения продукта могут рассматриваться только указанные четыре страны.
5. Перейдите на вкладку **Individuals**. Создайте еще один индивид класса *Страна*: *Россия*.
6. Запустите машину логического вывода. Убедитесь, что машина логического вывода не обнаружила противоречий, хотя индивид *Россия* не принадлежит к указанному списку в определении класса *Страна*. Это происходит, потому что соблюдается предположение об открытости мира, и если явно не указано, что индивид *Россия* различен с другими индивидами, то это не является фактом для машины вывода.
7. Укажите, что индивид *Россия* отличен от индивидов {Америка, Великобритания, Франция, Италия} (с помощью области **Different Individuals** окна **Description:Россия**) и снова запустите машину логического вывода. Убедитесь, что машина вывода выявила противоречивость онтологии, и верните онтологию в согласованное состояние.

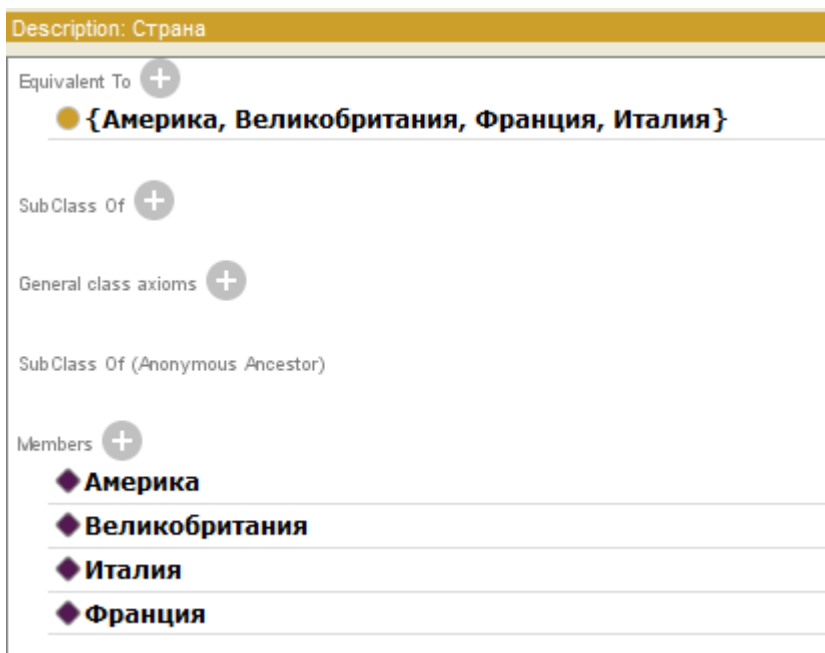


Рисунок 6.8. Окно Description:Страна

СПИСОК ЛИТЕРАТУРЫ

1. Гаврилова Т.А., Хорошевский Ф.В. Базы знаний интеллектуальных систем. СПб.: Питер, 2000. 382 с.
2. Стандарты семантического веба [Электронный ресурс]. Режим доступа: <http://www.w3.org/standards/semanticweb/>
3. Скворцов Н. А. Формирование глоссария по онтологическому моделированию: состояние и перспективы развития <http://synthesis.ipi.ac.ru/synthesis/publications/11ont-glos/11ont-glos.pdf>
4. Рабочая группа симпозиума «Онтологическое моделирование». URL: <http://ontology.ipi.ac.ru/>
5. OWL 2 Web Ontology Language. Document Overview (Second Edition). W3C Recommendation 11 December 2012 [Электронный ресурс]. Режим доступа: <http://www.w3.org/TR/2012/REC-owl2-overview-20121211/>
6. Сытник А.А., Вагарина Н.С., Мельникова Н.И. Введение в язык запросов к семантическим данным SPARQL: учеб. пособие. Саратов: Сарат. гос. техн. ун-т, 2015. 57 с.
7. Шульга Т.Э. Нурмамедов Н.Э. Анализ популярных редакторов веб-онтологий. // Инжиниринг предприятий и управление знаниями: сб. научных трудов 18-й Российской научно-практической конференции. М., 2015. С. 35.-38.
8. Официальный сайт редактора Protégé [Электронный ресурс]. Режим доступа: <http://protege.stanford.edu/>
9. Matthew Horridge et al. 2011. A Practical Guide To Building OWL Ontologies Using Protégé 4 and CO-ODE Tools, Edition 1.3, published by the University of Manchester, 24 Mar 2011, 108 pp. [Электронный ресурс]. Режим доступа: http://mowl-power.cs.man.ac.uk/protegeowltutorial/resources/ProtegeOWLTutorialP4_v1_3.pdf
10. Антонио Г., Грос П., Хармелен ван Ф., Хоекстра Р. Семантический веб: учеб. / пер. с англ. Т. Шульги. 3-е изд. М.: ДМК Пресс, 2016. 240 с.
11. Болотова Л. С. Системы искусственного интеллекта: модели и технологии, основанные на знаниях: учеб. / ФГБОУ ВПО РГУИТП ФГА ГНИИ ИТТ «Информика». М: Финансы и статистика, 2012. 664 с.
12. Gruber, Thomas R. (June 1993). A translation approach to portable ontology specifications (PDF). Knowledge Acquisition 5 (2): 199–220. doi:10.1006/knac.1993.1008. [Электронный ресурс]. Режим доступа: <http://tomgruber.org/writing/ontolingua-kaj-1993.pdf>

13. Linked Data Glossary. W3C Working Group Note 27 June 2013. [Электронный ресурс]. Режим доступа: <http://www.w3.org/TR/2013/NOTE-ld-glossary-20130627/#ontology>

14. Ной Наталья и МакГиннесс Дэбора Л. Разработка онтологий 101: руководство по созданию Вашей первой онтологии [Электронный ресурс]. Режим доступа: ifets.ieee.org/russian/depository/ontology101_rus.doc

Учебное издание

СЫТНИК Александр Александрович
ШУЛЬГА Татьяна Эриковна
ВАГАРИНА Наталья Сергеевна
МЕЛЬНИКОВА Нина Ивановна

ОСНОВЫ ПОСТРОЕНИЯ OWL-ОНТОЛОГИЙ
С ИСПОЛЬЗОВАНИЕМ РЕДАКТОРА
PROTÉGÉ DESKTOP 5.0 БЕТА

Учебное пособие

Редактор Л.А. Скворцова

Компьютерная верстка Т.В. Семеновой

Подписано в печать 01.10.15

Формат 60×84 1/16

Бум. офсет

Усл. печ. л. 5,25

Уч.-изд. л. 4,8

Тираж 100 экз.

Заказ 160

С 59

Саратовский государственный технический университет

410054, Саратов, Политехническая ул., 77

Отпечатано в Издательстве СГТУ. 410054, Саратов, Политехническая ул., 77

Тел.: 24-95-70; 99-87-39, e-mail: izdat@sstu.ru

